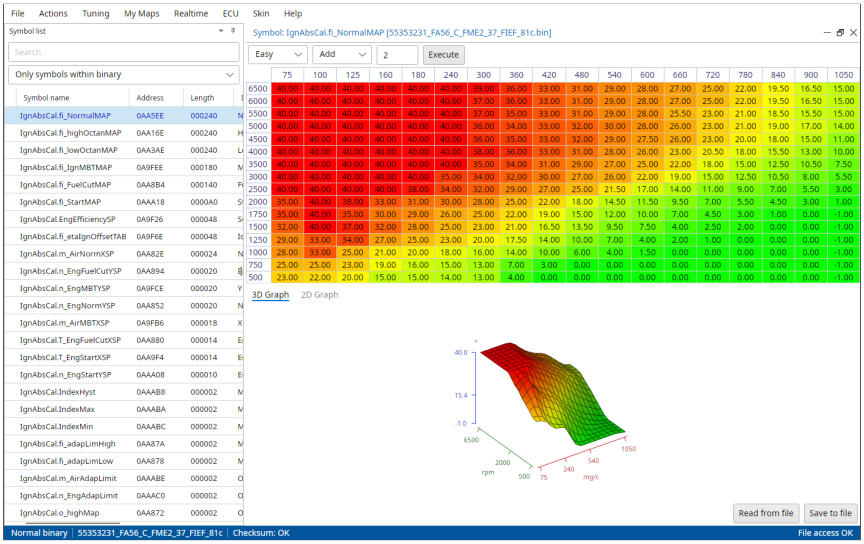
# TxSuite user manual

Trionic 8



# Table of contents

Getting started . . . 5
Documentation . . . 5
Installation . . . 5
Windows . . . 5
Linux . . . 5
macOS . . . 6
Supported interfaces . . . 6
Settings and log files . . . 6
Coming from the old suites . . . 6
Updates . . . 7
Disclaimer . . . 7
The main window . . . 8
The menu . . . 8
The ribbon bar . . . 8
Opening a file . . . 9
Dropping files . . . 10
Windows in the workspace . . . 10
The status bar . . . 10
Symbol palette and command palette . . . 10
Keyboard shortcuts . . . 12
The symbol list . . . 13
Selecting symbols . . . 13
Searching . . . 13
Filtering . . . 14
Sorting and grouping . . . 14
Map preview popup . . . 15
The symbol menu . . . 16
Hiding the symbol list . . . 16
Working with maps . . . 17
Hexadecimal mode . . . 17
Color indicators . . . 17
Selecting cells . . . 17
Adjusting values in a map . . . 17
Copy and paste . . . 18
Smoothing . . . 18
Editing the axes . . . 18
Graphs . . . 19
Saving changes . . . 19
Maps from the ECU . . . 19
Checksum . . . 20
File menu . . . 21
File → Open file... . . . 21
File → Recent . . . 21
File → Save all . . . 21
File → Save as... . . . 21
File → Export to S19 . . . 21
File → Create backup file . . . 21
File → Import XML descriptor, AS2 descriptor and CSV descriptor . . . 21
File → Import tuning package . . . 21
File → Edit a tuning package . . . 22
File → Create binary from TIS file . . . 22
File → Setup log filters . . . 22

File → Generate Idc file . . . 22
File → Settings . . . 22
File → Lookup partnumber . . . 23
File → Project . . . 24
File → Exit . . . 24
Projects . . . 25
File → Project → Create a project . . . 25
File → Project → Open a project . . . 25
File → Project → Close project . . . 26
File → Project → Show transaction log . . . 26
File → Project → Roll back/undo . . . 26
File → Project → Roll forward/redo . . . 26
File → Project → Rebuild file . . . 26
File → Project → Edit project . . . 27
File → Project → Add note to project log . . . 27
File → Project → Show project logbook . . . 27
File → Project → Produce latest binary . . . 27
Settings . . . 28
Settings → Common . . . 28
User interface settings . . . 28
Project settings . . . 29
Realtime settings . . . 29
Settings → Logging . . . 30
Trionic 8 settings . . . 32
Settings → T8 . . . 32
User interface settings . . . 32
Realtime settings . . . 32
Actions menu . . . 33
Actions → Verify checksum . . . 33
Actions → Firmware information . . . 33
Actions → VIN decoder . . . 33
Actions → Browse axis information . . . 33
Actions → Show interrupt vectors . . . 34
Actions → Show disassembly . . . 34
Actions → Show full disassembly . . . 34
Actions → Compare symbols with other binary . . . 34
Actions → Compare binary outside symbolrange . . . 35
Actions → Compare binary with other binary . . . 35
Actions → Transfer maps to another binary . . . 36
Actions → Copy address table to another binary . . . 37
Actions → Search map content . . . 37
Actions → Export map to CSV . . . 37
Actions → View file in hex . . . 37
SRAM snapshots . . . 38
Actions → Import SRAM snapshot . . . 38
Firmware information . . . 39
Trionic 8 tools . . . 40
Actions → Airmass result viewer . . . 40
Actions → TEM editor . . . 41
Actions → PID editor . . . 41
Bit mask symbols . . . 42
Tuning menu . . . 43
Tuning → Tuning Wizard . . . 43
Tuning → Airmass . . . 43

Tuning → Torque . . . 44
Tuning → Injectors . . . 44
Tuning → Fuel . . . 44
Tuning → Boost . . . 44
Tuning → Ignition . . . 44
Tuning → Pedal . . . 44
My Maps menu . . . 45
My Maps → Define myMaps... . . . 45
Realtime menu . . . 46
Realtime → Connect ECU . . . 46
Realtime → Toggle realtime panel . . . 46
Realtime → Export logfile to LogWorks . . . 47
Realtime → Load logfile . . . 47
Realtime → View matrix from logfile . . . 48
Realtime → Set symbol colors . . . 48
Realtime → Get fault codes . . . 48
Realtime → Write log marker . . . 49
Realtime → Export logfile to CSV . . . 49
Trionic 8 realtime . . . 50
Realtime → Get ECU information . . . 50
Realtime → Clear DTC and knock counters . . . 50
Realtime → View knock count map . . . 50
Realtime → View misfire tab . . . 50
ECU menu . . . 51
ECU → Read ECU . . . 51
ECU → Flash current file to ECU . . . 51
ECU → Recover ECU . . . 51
Skin menu . . . 52
Skin → Light, Dark and Follow the system . . . 52
Skin → Cascade windows and Tile windows . . . 52
Skin → Dock all floating windows . . . 52
Help menu . . . 53
Help → T5, T7 and T8 . . . 53
Help → Release notes . . . 53
Help → Check for updates . . . 53
Help → About TxSuite . . . 53

# Getting started

TxSuite is T5Suite, T7Suite and T8Suite in one program. The three suites were rebuilt on .NET 10 and Avalonia, without the commercial components the old suites needed, so TxSuite runs on Windows, Linux and macOS. Open a T5, T7 or T8 binary and TxSuite works on it the way that ECU's suite did: the same files read, edited and written, the same maps, tools and realtime panel. It talks to the ECU through TrionicCANLib, the library of the Trionic CAN Flasher.

This manual covers TxSuite with a Trionic 8 binary. Trionic 5 and Trionic 7 have a manual of their own, Help → T5 → User manual and Help → T7 → User manual.

What all three ECU's have in common is written the same way in all three manuals. The parts only your ECU has are in chapters of their own, right after the chapter they belong to.

## *Documentation*

This manual tells you what the program does. What the ECU does, its hardware, firmware and maps and how to tune them, is in Dilemma's Trionic documentation.

For the T8 that is "Analyzing Trionic 8 with TxSuite", Help → T8 → Trionic 8 documentation.

Both are kept up to date in one place, https://github.com/roffe/TrionicDocumentation. Found something wrong or missing? Open an issue or send a PR there.

## *Installation*

The newest TxSuite and TxExtras are on one page, https://github.com/roffe/TuningSuites/releases/latest. All downloads are on the releases page: the `Txsuite_v…` releases, and `Txsuite_nightly` with the latest build.

The stock binaries of all three ECU's are a separate download, TxExtras. TxSuite uses them wherever it needs the original file of a partnumber, Lookup partnumber for example. They go into `Binaries\T5`, `Binaries\T7` and `Binaries\T8` in TxSuite's folder.

### *Windows*

Run TxSuiteSetup.exe. It installs to `Program Files (x86)\MattiasC\TxSuite`, next to any old T5Suite, T7Suite or T8Suite (it replaces none of them), and asks whether `.bin` files should open in TxSuite. Then TxExtrasSetup.exe puts the stock binaries into TxSuite's folder. Running the setup of a newer build over an existing installation upgrades it.

Rather not install it? TxSuite-win-x86.zip runs from wherever you unpack it, unpack TxExtras.zip into the `TxSuite` folder for the stock binaries. It doesn't open `.bin` files by itself, and its update check opens the release page instead of downloading the setup.

### *Linux*

Download TxSuite-linux-x64.tar.gz (or -linux-arm64) and run it from where you extracted it. Unpack TxExtras.zip into the `TxSuite` folder for the stock binaries.

```
tar xzf TxSuite-linux-x64.tar.gz
./TxSuite/TxSuite
```

`./TxSuite/install-desktop.sh` adds TxSuite to the application menu and to Open with for `.bin` files, for your user only. Run it again after moving the folder.

The CombiAdapter needs libusb and the udev rule next to the program:

```
sudo cp TxSuite/70-txsuite.rules /etc/udev/rules.d/
sudo udevadm control --reload && sudo udevadm trigger
```

There is also an AppImage, all of TxSuite in one file (TxSuite-linux-x64.AppImage, or -linux-arm64). `chmod +x` it and start it. It needs FUSE, which most desktops have; without it start it with `--appimage-extract-and-run`. The stock binaries go next to the AppImage, unpack TxExtras.zip in its folder. The udev rule isn't in the AppImage, take `70-txsuite.rules` from the tar.gz.

### *macOS*

Download TxSuite-osx-arm64.zip (Apple silicon) or -osx-x64 (Intel). The build is not signed, so clear the quarantine flag after extracting it, then start it from Terminal or by double-clicking TxSuite in Finder:

```
unzip TxSuite-osx-arm64.zip
xattr -dr com.apple.quarantine TxSuite
./TxSuite/TxSuite
```

## *Supported interfaces*

TxSuite supports the same interfaces as the Trionic CAN Flasher. On a T5 it talks to the ECU's P-bus.

| **Interface** | **Windows** | **Linux** | **macOS** |
|---|---|---|---|
| Lawicel CANUSB | Lawicel CANUSB DLL driver | FTDI serial port | FTDI serial port |
| CombiAdapter | libusb (WinUSB driver) | libusb + udev rule | libusb |
| OBDLink SX / EX, ELM327 | serial port | serial port | serial port |
| Just4Trionic | serial port | serial port | serial port |
| SLCAN | serial port | serial port | serial port |
| Kvaser HS | Kvaser CANlib driver | Kvaser linuxcan | not supported |
| J2534 (Beta) | vendor DLL (32-bit) | vendor `.so` | not supported |
| Wideband lambda (realtime) | serial port | serial port | serial port |

The adapters still need their own drivers, see Adapter setup in the flasher's README, https://github.com/roffe/Trionic#adapter-setup. Which adapter TxSuite uses is set in File → Settings → Common.

## *Settings and log files*

Settings and log files are in `%APPDATA%\MattiasC\TxSuite` on Windows and `~/.config/MattiasC/TxSuite` on Linux and macOS. File → Settings → Logging decides which log files get written, at what level and for how long, and its Open button shows the folder on any system.

The first start takes over the settings of T5Suite, T7Suite and T8Suite once, with their Recent lists merged into one. After that TxSuite keeps its own settings and the old suites are left alone.

## *Coming from the old suites*

TxSuite looks and works like the old suites, with some differences:

- One window for all three ECU's. The file you open decides the ECU.

- One menu bar instead of the ribbon. Want the ribbon back? File → Settings → Common → Show ribbon bar instead of menus.
- One settings dialog with a tab per ECU.
- Excel export and import are CSV now, and the map tables copy as text that pastes into Excel.
- Autologging, sound notifications, full screen and the panel snapshot (F9) are gone.
- New are the symbol and command palettes, colour blind modes, a dark skin, dropping files on the window and multi-step undo in the map viewer.

## *Updates*

TxSuite looks for a newer release when it starts and from Help → Check for updates, and shows the result in the status bar. When there is one it asks. OK downloads the setup on Windows and opens the release page elsewhere (and for the Windows zip).

## *Disclaimer*

This is Open Source software that pokes around in your car's control system. The authors of the tools shall not be held accountable for how you decide to use the tools. If you are not careful, you can easily brick your car with these tools so please use this software with care.

# The main window

TxSuite is controlled by the menu and the symbol list shown on the left hand side of the screen. Maps, compare results, the realtime panel and the other tools open as windows in the workspace on the right. The status bar at the bottom shows what is open and how it's doing.

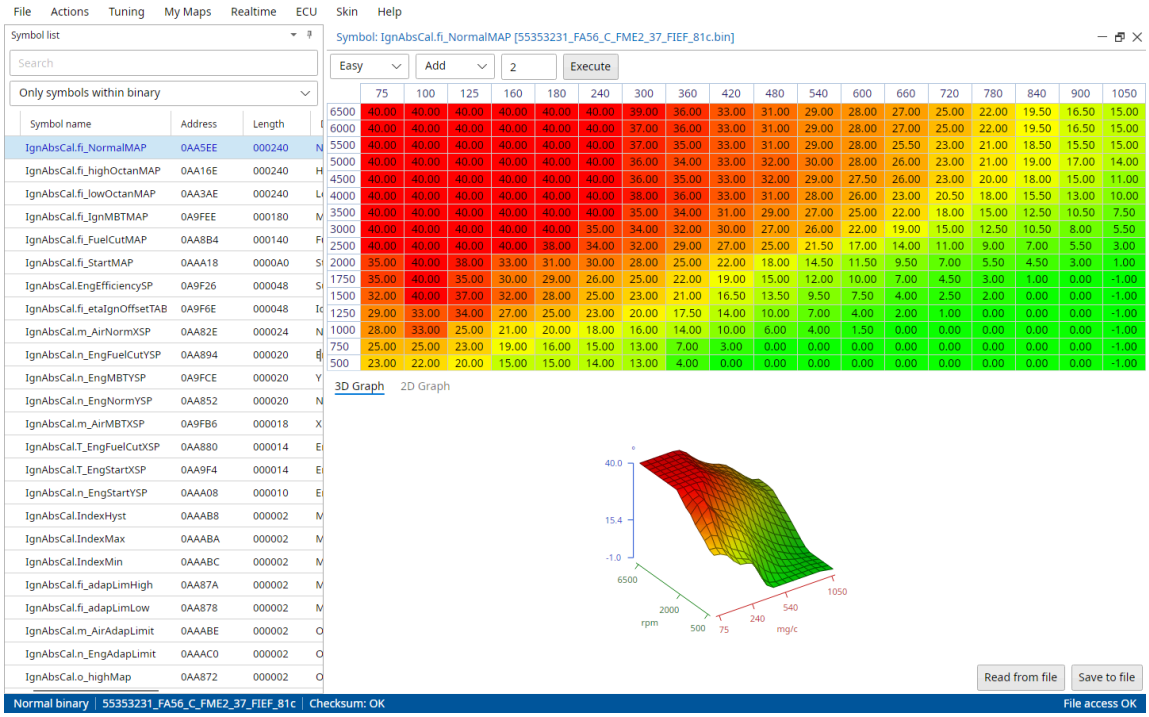



**Image 1: The main window with a Trionic 8 ignition map open**

## *The menu*

The menus are File, Actions, Tuning, My Maps, Realtime, ECU, Skin and Help. What every ECU has is always there. An ECU's own items only show while a file of that ECU is open, in the place they had in that ECU's old suite. With no file open only the shared items show.

Items that need an open file are greyed out until you open one. The ECU follows the file as well: Connect ECU, Read ECU and the other ECU actions need a file of that ECU open, because the file tells TxSuite which ECU it is talking to. Without one TxSuite says "A file must be loaded first: it tells which ECU (Trionic 5, 7 or 8) to connect to."

The menus are described one by one in the chapters after this one.

## *The ribbon bar*

The old suites had a ribbon bar instead of menus. File → Settings → Common → Show ribbon bar instead of menus brings it back, at once.

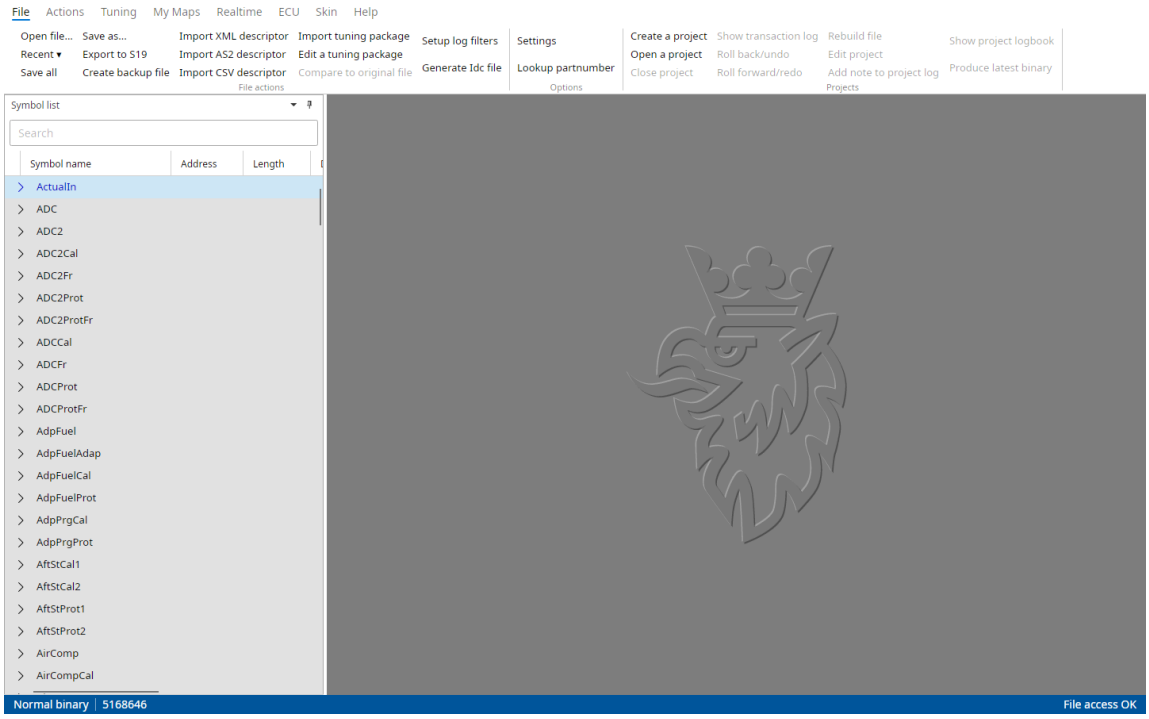



**Image 2: The ribbon bar, File tab**

The ribbon is made from the menus, so it always has the same items. Each menu is a tab, each section of a menu a group with its items as buttons. Groups that don't fit in the window fold into a button that drops them down.

- Double-click a group's caption to fold it, double-click a folded one to keep it open.
- Drag a group by its caption to put it somewhere else on its tab.
- Right-click a caption and pick Reset this tab's groups to get the menu's order back.

The folds and order are remembered per ECU and per tab. The buttons have text only, no icons, and there is no Exit button: close the window.

## *Opening a file*

File → Open file… opens a binary or a Motorola S19 file. TxSuite looks at the file and picks the ECU by it: T5 and T7 by size and content, T8 by its 1 MB size.

A file opens from these places too:

- File → Recent, the files opened before.
- A file dropped on the main window.
- Open with in the file manager, or the file name on the command line.
- Opening a project, or Lookup partnumber's Open this file and Create new from original.
- File → Settings → Common → Auto load last file on startup reopens the last file (or project) when TxSuite starts.

Opening a file of another ECU closes everything of the one before, as when the suites were separate programs. Maps with unsaved changes are asked about first, Cancel keeps it all. Nothing switches while an ECU is being flashed or a T5 autotune runs. A file no ECU takes gives "File is not a Trionic 5, 7 or 8 binary file!".

After opening, the symbol list on the left is filled with the symbols of the file, see the next chapter.

## *Dropping files*

Files dropped on the main window load like they do from the menu:

- a binary opens as with File → Open file,
- an SRAM dump loads as with Actions → Import SRAM snapshot,
- a logfile of the open file's ECU opens as with Realtime → Load logfile.

Everything else (tuning packages, descriptors, CSV files, reports) stays with its menu item, so nothing changes the binary by accident. A file dropped on a hex view opens in that hex view.

## *Windows in the workspace*

Maps, compare results, the hex view, the realtime panel and the other tools open as windows inside the workspace. Move them by their title bar, resize them by their edges, and minimize, maximize or close them with the buttons on the right of the title bar. A double-click on the title bar maximizes a window too.

Drag a window's title bar out of the main window and it floats in a window of its own, handy with a second screen. Its "Dock back into main window" button brings it back.

The Skin menu has the window layout: Cascade windows, Tile windows horizontally, Tile windows vertically and Dock all floating windows.

When a compare shows a map of two files, both share one window with a tab per file.

## *The status bar*

The status bar shows, from left to right:

- "Open/dev binary" for open (development) software, else "Normal binary",
- the file name and "Checksum: OK" or what is wrong with it,
- the SRAM snapshot that is loaded, if any,
- the connection to the ECU,
- the result of the update check.

The right side shows the progress of what is running (reading, flashing, extracting) and whether the file can be written, "File access OK".

## *Symbol palette and command palette*

Like in VS Code, Ctrl+P opens a symbol by any part of its name and Ctrl+Shift+P runs any menu command (Cmd on macOS). Type a part of what you are looking for, move with the arrow keys, Enter or a click picks, Escape closes.

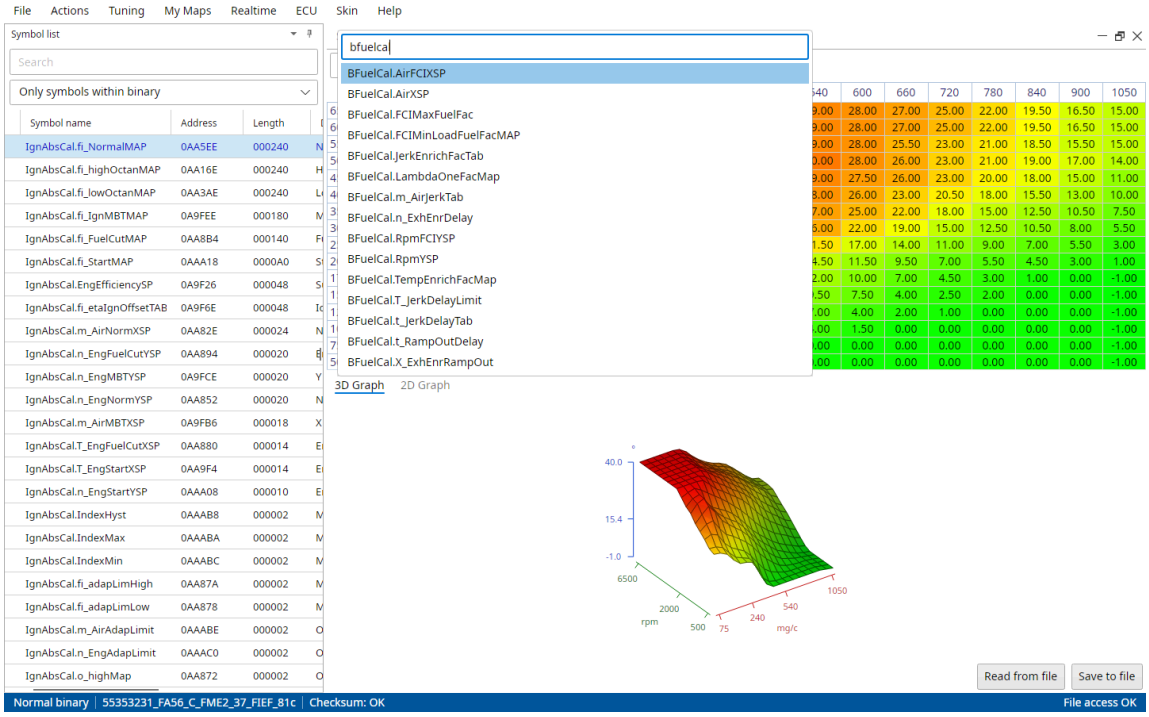



**Image 3: Ctrl+P, the symbol palette**

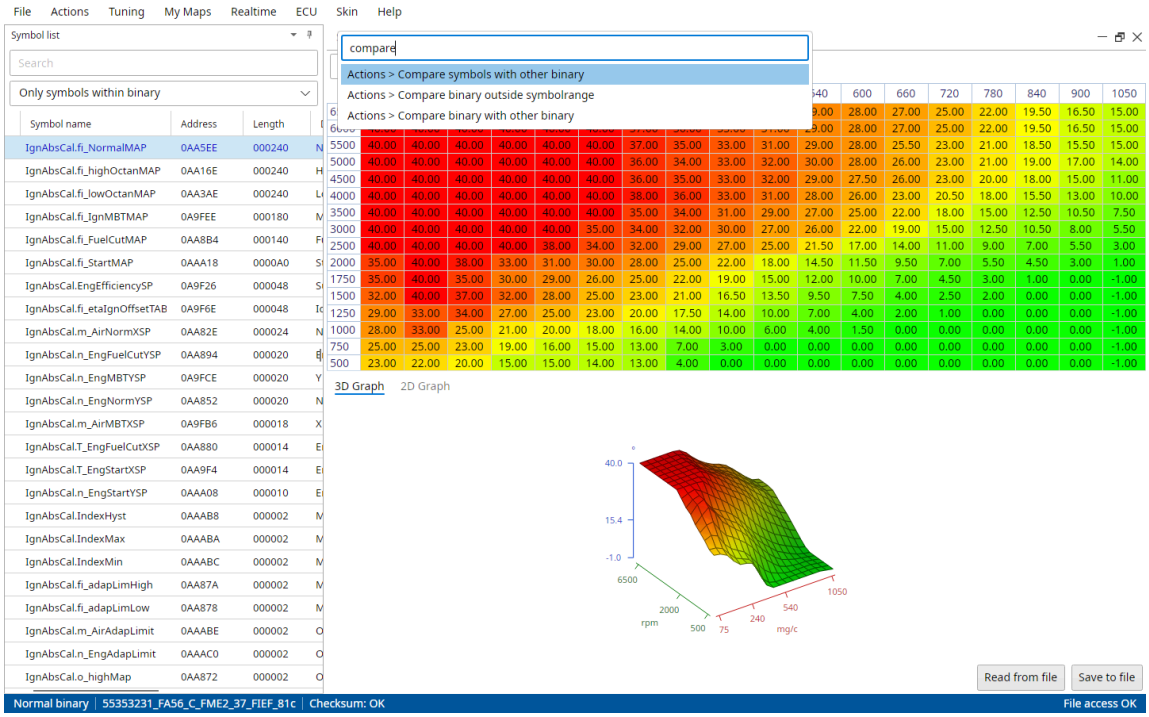



**Image 4: Ctrl+Shift+P, the command palette**

The symbol palette lists the symbols of the open file that a double-click in the symbol list would open. The command palette lists every menu command that is available right now as “Menu > Item”, the Recent files and the map entries of the Tuning and My Maps menus included.

## *Keyboard shortcuts*

| Key | What it does |
|---|---|
| Ctrl+P | Open a symbol by name |
| Ctrl+Shift+P | Run a menu command |
| Shift+F1 | Realtime → Toggle realtime panel |
| F6 | Realtime → Write log marker |
| Enter | Open the selected symbol in the symbol list |

The map viewer has its own keys, see Working with maps.

# The symbol list

After a file is opened the symbol list on the left hand side of the main window is filled with its symbols. The symbols are grouped by category and every category starts folded, click the arrow in front of one to open it. The Left arrow key folds the category of the selected symbol again.

TxSuite reads the symbol table from the file, packed or encrypted, and checks the checksum while it's at it. Binaries without a symbol table can be described with an XML, AS2 or CSV descriptor, see the File menu. With File → Settings → T8 → Auto mapdetection active TxSuite tries to name the symbols of such a file by itself.

The columns are Symbol name, Address, Length, Description and User description. More are hidden, like Number, SRAM address and Category, the column chooser shows them (see Sorting and grouping below). Addresses and lengths are hexadecimal or decimal by File → Settings → Common → Show addresses and lengths in Hex.

The User description column is yours: click a cell to type a note for that symbol. It is saved next to the bin in `<bin>.xml`.

With File → Settings → Common → Show descriptions under symbol names each symbol's description shows on a line of its own under its name, as the old T7Suite and T8Suite did. File → Settings → Common → Symbol list font size makes the list's text larger or smaller.

## *Selecting symbols*

Choose a symbol by double-clicking it or by selecting it and pressing Enter. A new window opens in the workspace with the map of that symbol, titled "Symbol: <name> [<file>]". Opening a map that is open already brings its window to the front. Working with that window is the next chapter.

It also works the other way around: the map in front is selected in the symbol list, with its category opened.

Ctrl+P is the quick way, type any part of a symbol's name and Enter opens it.

## *Searching*

Type in the Search box above the list and it shows only the symbols with that text in their name, description, user description, address or length. The categories open so the matches show. Clear the box and the categories fold again, except the one of the selected symbol.

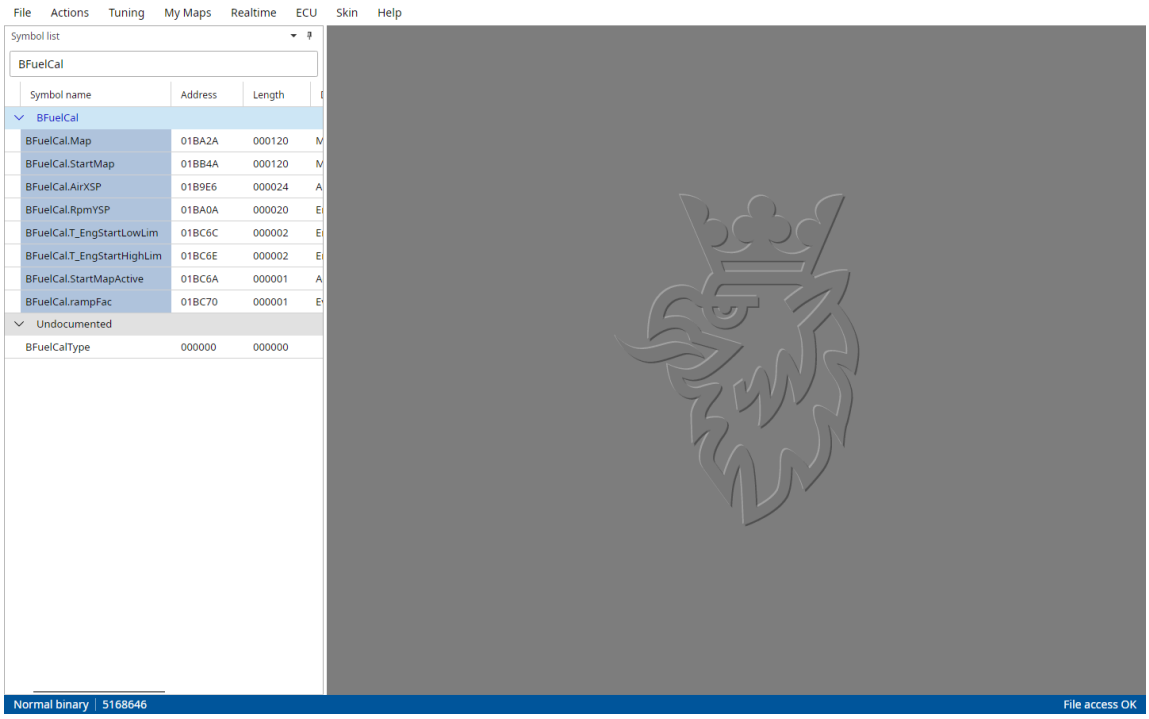



**Image 5: Searching the symbol list**

## *Filtering*

Below the Search box is a filter. By default it shows “Only symbols within binary”, the symbols that are actually stored in the flash. “All symbols” shows the symbols that only live in SRAM too, and “Only live-tuneable symbols” only the ones you can change in the running ECU.

Right-click a column header and choose Show auto filter row for a filter box per column. Text typed in one shows only the symbols with that text in that column, and several boxes work together.

## *Sorting and grouping*

Click a column header to sort on that column, click it again to sort the other way around. The categories stay in their order, the symbols are sorted within them.

Right-click a column header for more:

- Sort ascending, Sort descending and Clear sorting.
- Group by this column, Group by category (the default) and No grouping.
- Hide this column, and the Column chooser to show or hide any column.
- Best fit and Best fit (all columns) to size the columns to their content.
- Show auto filter row, see Filtering.

Columns can be dragged to another place and resized by their edges.

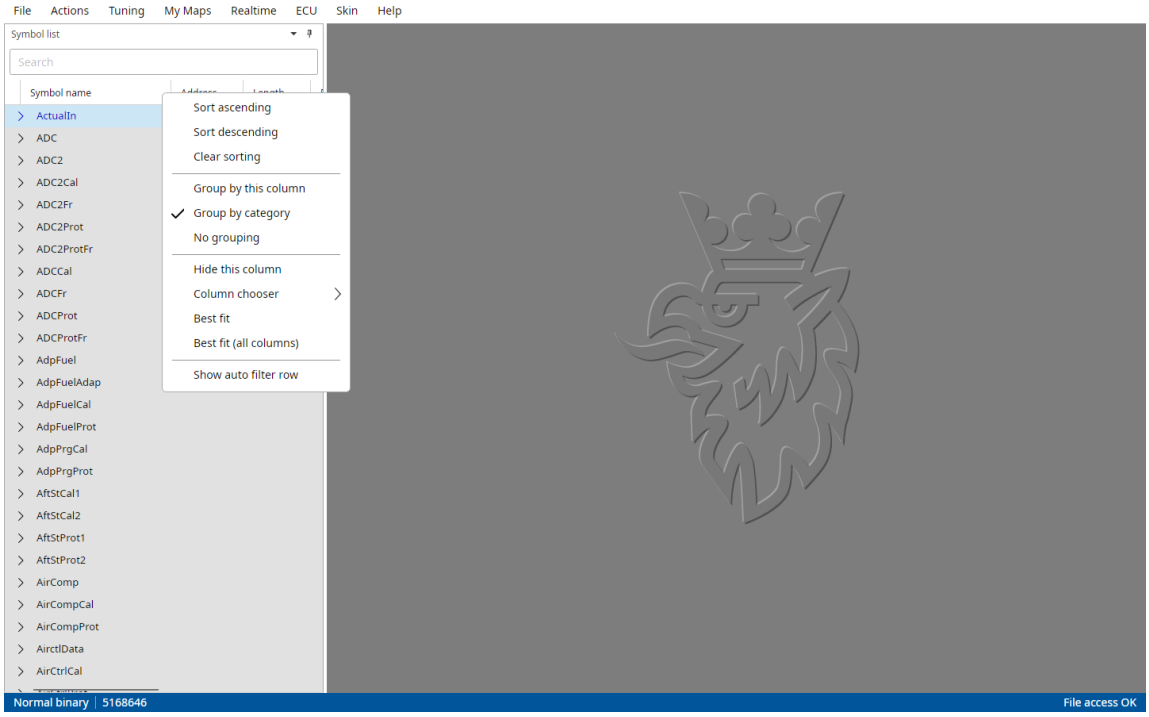



**Image 6: The column header menu of the symbol list**

## *Map preview popup*

Point at a symbol's name and its map pops up as a read-only table next to it. File → Settings → Common → Show map preview popup turns it off and on.

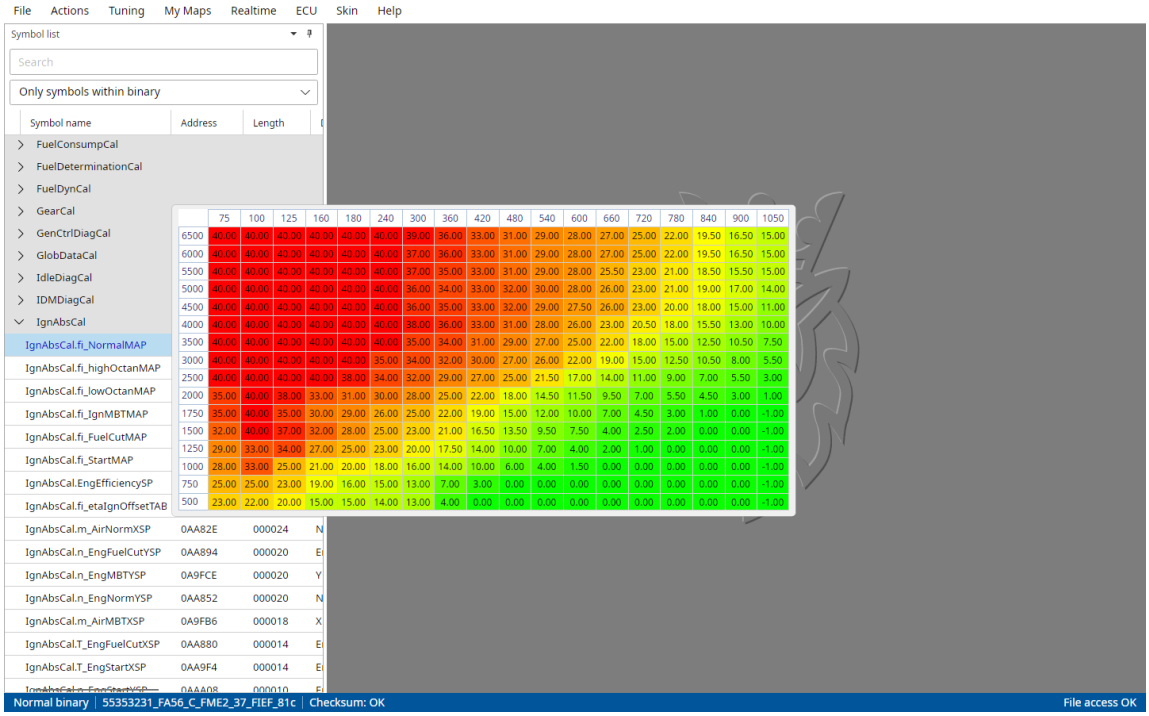



**Image 7: The map preview popup of a T8 ignition map**

## *The symbol menu*

Right-click a symbol for these:

**Add to realtime list**
Adds the symbol to the realtime panel's free logging list.

**Add to MyMaps**
Adds the symbol to your own My Maps menu.

**Read from SRAM file**
Opens the symbol's map from the SRAM snapshot that is loaded with Actions → Import SRAM snapshot.

**Browse axis information**
Shows the symbol in Actions → Browse axis information.

**Export symbollist as CSV**
Saves the whole symbol list as a CSV file.

**Export as tuning package**
Saves the selected symbols as a tuning package.

**Export fixed tuning package**
Saves a tuning package with the maps a stage tune touches.

**Edit a tuning package**
Opens the tuning package editor, as File → Edit a tuning package. Its Add selected symbols button takes the symbols selected here.

## *Hiding the symbol list*

File → Settings → Common → Hide symbol window hides the symbol list to a tab at the side of the window. The tab brings the list out again, and it slides away when you open a map. The pin button at the top of the list does the same.

# Working with maps

A symbol opened from the symbol list shows in a map viewer, a window in the workspace with the table on top and its graphs below. You can see one in the picture at the start of The main window chapter.

From top to bottom the map viewer has:

- the toolbar: the view type, and the math on the selection (operation, value and Execute),
- the table, with the x axis above it and the y axis on its left,
- the graphs, a 3D Graph tab and a 2D Graph tab,
- the buttons Read from ECU, Save to ECU, Read from file and Save to file.

The splitter between the table and the graphs moves up and down. File → Settings → Common → Show graphs in mapviewer turns the graphs off, for slower computers. Read from ECU and Save to ECU only show for symbols the ECU has in its RAM.

## *Hexadecimal mode*

The view type box in the toolbar switches the table between Hex, Decimal, Easy and ASCII.

**Hex**
: The raw values in hexadecimal.

**Decimal**
: The raw values in decimal, as the ECU stores them.

**Easy**
: The values converted with the map's factor and offset to what they mean, degrees, bar, mg/c and so on. Easy view is best if you don't know what this means ;)

**ASCII**
: The bytes as text, read only.

The view type a map opens in is set by File → Settings → Common → Default view type for maps. Each map viewer can still be switched to another view type.

## *Color indicators*

All maps are displayed in colours from green (low values) through yellow to red (high values), over the smallest and largest value of that map. If you find this hard to tell apart, File → Settings → Common → Color blind mode has other colour scales: Universal, Protanopia, Tritanopia and Deuteranomaly. They are used for the table, both graphs and the airmass result viewer, open map viewers change at once.

Maps opened from the ECU while connected keep their own colours, so you can see they show what's live in the ECU.

File → Settings → Common → Mapviewer cell spacing puts space between the cells, 0 to 20 pixels. With a space the cells show as separate tiles without the lines between them.

## *Selecting cells*

Click a cell to select it, drag to select a block. Shift+click extends the selection, Ctrl+click adds or removes cells. The arrow keys move the selection, Ctrl+A selects the whole map.

## *Adjusting values in a map*

To avoid that you have to adjust all values of a "large" map manually some features have been added to the map editor:

| Key | What it does to the selected cells |
|---|---|
| Plus | adds 1 |
| Minus | subtracts 1 |
| PageUp | adds 10 (0x10 in hex view) |
| PageDown | subtracts 10 (0x10 in hex view) |
| Home | sets them to the maximum value |
| End | sets them to 0 |

Typing a value starts editing, F2 edits the focused cell starting from its value, Enter stores it and Esc throws it away. With several cells selected the typed value goes into all of them.

The toolbar does math on the selection: pick Add, Multiply, Divide or Fill, type the value and click Execute.

Ctrl+Z undoes the last change and Ctrl+Y redoes it, as many steps back as you like. Undo and Redo are in the table's right-click menu too.

## *Copy and paste*

To be able to get your work done faster you can select one or more cells in a table and copy them to the clipboard by right-clicking and selecting "Copy selected cells". With nothing selected the whole map is copied.

To paste the cells select the location where you want the cells to appear (this could be in another map and even in another binary), right-click and select "Paste selected cells" and then "At original position" or "At currently selected location".

Copy selected cells uses TxSuite's own format, for pasting into maps. Ctrl+C copies the selected cells as plain text with tabs, that pastes straight into a spreadsheet.

## *Smoothing*

Select a row, a column or a block and pick Smooth selection in the table's right-click menu. A row or column becomes a straight line between its first and last value, a block gets the average of its neighbours in each cell inside it, the cells on its edge stay as they are.

## *Editing the axes*

Edit x-axis and Edit y-axis in the table's right-click menu open the axis as a map of its own, to change its values.

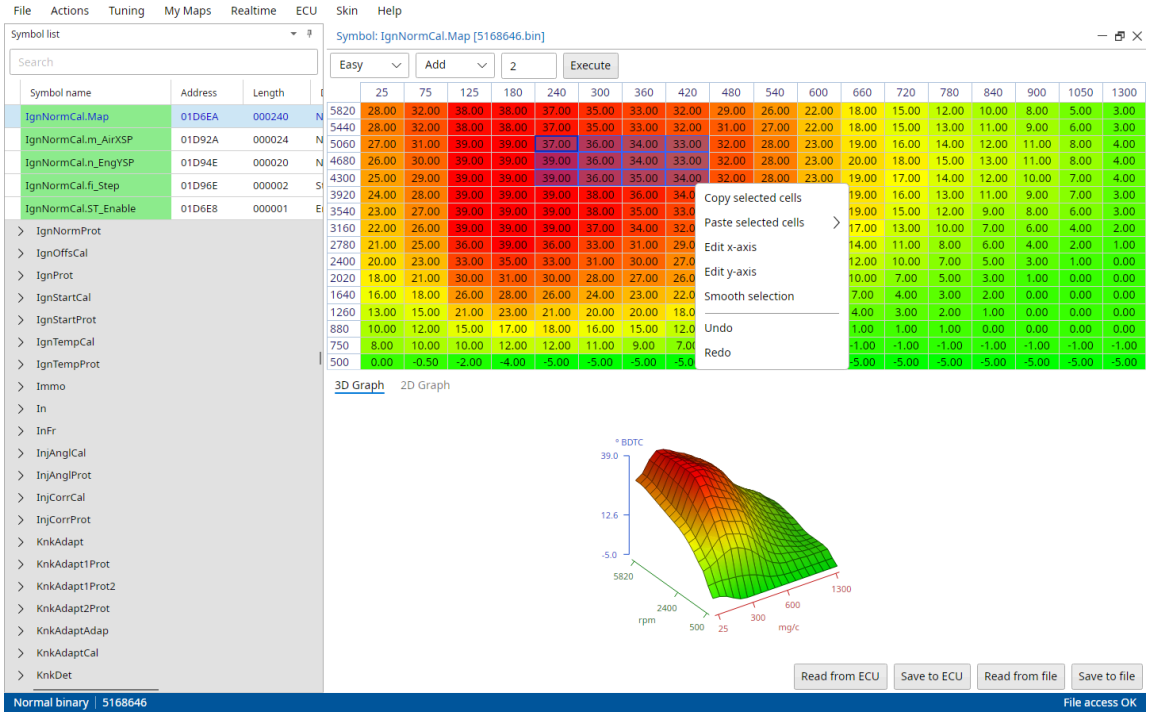



**Image 8: The table's right-click menu over a block of cells**

## *Graphs*

The 3D Graph tab shows the map as a surface. Drag with the left mouse button to turn it, with the right button to roll it and with the middle button to move it, and the mouse wheel zooms.

The 2D Graph tab shows one column of the map as a line over the y axis, the slider below it picks which column.

With File → Settings → Common → Synchronize mapviewers on, viewers of the same map follow each other: selecting cells in one selects them in the others, and turning one graph turns the others.

## *Saving changes*

Save to file writes the map into the binary and updates the checksum. Read from file throws away the changes and reads the map from the file again.

Closing a viewer with changes asks "Data was mutated, do you want to save these changes in you binary?", Yes saves, No throws the changes away and Cancel keeps the viewer open. Closing TxSuite asks the same for each changed viewer. File → Save all saves every open viewer at once.

In a project every save is a transaction in the project's log, see the Projects chapter.

## *Maps from the ECU*

When the ECU is connected, Read from ECU reads the map from the ECU's RAM and Save to ECU writes it there, every viewer of that map follows. When the ECU refuses a write TxSuite says so.

Maps that only live in the ECU's RAM are read from the ECU when you open them. Offline they open from the SRAM snapshot loaded with Actions → Import SRAM snapshot.

File → Settings → Common → Auto update SRAM viewers every … seconds reads these maps from the ECU again and again while connected. This enables you to keep an eye on for example the

knock map. Viewers with unsaved changes are left alone. Don’t set the interval too short, it takes time from the realtime panel.

## *Checksum*

For you to be able to use the binary after altering maps or other information within it, the checksum must be updated after alteration of data. TxSuite does that every time it writes a map into the binary.

File → Settings → Common → Auto update checksum makes sure the checksum is right everywhere else too. Advised is to keep this turned on! With it off TxSuite asks before it corrects a wrong checksum, and before it flashes a file with one.

Actions → Verify checksum checks the open file’s checksum by hand.

On a T8 the checksum is checked every time a file opens, the status bar shows “Checksum: OK”. With Auto update checksum on a wrong one is corrected without asking, otherwise TxSuite asks per layer.

# File menu

The File menu opens, saves, imports and exports files, and holds Settings, Lookup partnumber and the Project submenu.

## *File → Open file…*

Lets you choose a binary file on your computer, or a Motorola S19 file which is converted first. TxSuite picks the ECU by the file and extracts all the relevant information from it, see Opening a file in The main window chapter.

## *File → Recent*

Shows you a list of recently opened files for easy access, of all three ECU's in one list.

## *File → Save all*

Normally TxSuite saves a map when you close its viewer and say Yes. If you have map viewers with changes open you can save all of them to the file at once with this option.

## *File → Save as…*

This lets you choose a different location and/or filename for the binary file you currently have open. Then TxSuite asks whether to open the copy, Yes and you work on the copy.

## *File → Export to S19*

This allows you to save the binary file in Motorola S19 format which can be used to program the ECU with a BDM interface.

## *File → Create backup file*

Lets you create a backup file for the binary at this point. It is wise to create backups before you start to make big changes to your file. If a project is opened, the backup file will be stored within the project folder, otherwise it will be stored in the folder where the bin file is located.

## *File → Import XML descriptor, AS2 descriptor and CSV descriptor*

Lets you import the symbol names for a binary from an XML, AS2 or CSV file. Binaries without a symbol table can be user-described, and if you have a descriptor for your binary file you can import it with these. The names show in the symbol list at once and are saved next to the bin in `<bin>.xml`. The Tuning menu is rebuilt with them.

## *File → Import tuning package*

Lets you import a tuning package which holds generic map information. TxSuite writes the package's maps into the open file and shows what it did in Import results. A map whose length in the package differs from the file's is not written.

T8 tuning packages are `.t8p` files.

## File → Edit a tuning package

If you want to remove maps from a tuning package or you want to add maps, you can edit the tuning package with this option. It starts empty. Open package... loads the maps of a package, Add selected symbols adds the symbols selected in the symbol list, Remove (or Del) drops rows. Each row's map opens in a map viewer that saves into the row, and the result is saved as a new package.

## File → Create binary from TIS file

SAAB has stored Trionic 8 application files in its Technical Information System (TIS). These files have an extension .gbf and they only contain a part of the complete binary file. To make things worse, these files are encrypted as well. Lucky for you, TxSuite can decrypt and build a new binary file for you based on another binary file and a compressed gbf file.

The new file gets the bootloader and adaption data of the base binary and the program of the TIS file. A `.gbf` or `.s19`, gzipped or not, is taken.

## File → Setup log filters

The realtime panel logs everything it reads. With log filters you tell TxSuite which lines of a log to keep, for example only the lines above a certain rpm, so you only look at what's relevant.

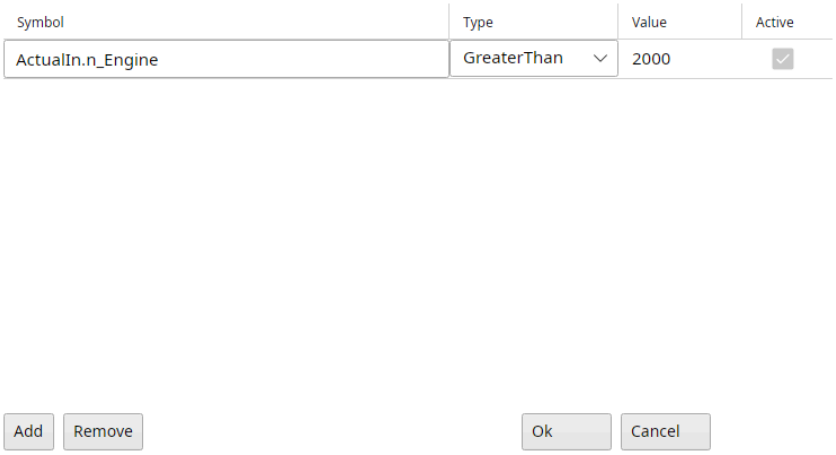



**Image 9: File → Setup log filters with one filter**

Each filter has a symbol, a type (GreaterThan, SmallerThan or Equals), a value and whether it's active. A line is left out when an active filter doesn't hold for it. The filters are applied in the log viewer and in both log exports, the logfile itself keeps everything.

## File → Generate Idc file

Writes an IDC script with the symbol names of the open file, for disassembling the binary in IDA Pro. If you're not sure what that means, you don't need this.

## File → Settings

This option lets you alter the settings that control the behaviour of the application. They have a chapter of their own, Settings.

## *File → Lookup partnumber*

Lets you look up a partnumber and the stock binary that belongs to it. It finds the partnumbers of all three ECU's, whichever file is open.

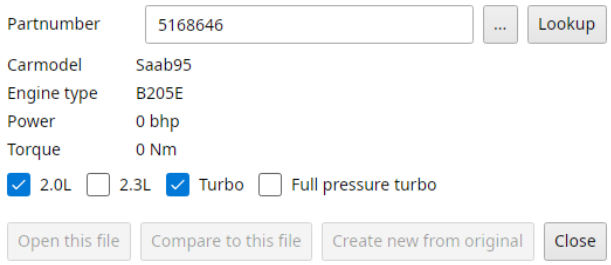



**Image 10: File → Lookup partnumber, 5168646 looked up**

In this screen you can either enter a known partnumber and click Lookup, or click the "..." button to browse all known partnumbers. TxSuite shows the car model and engine type the partnumber belongs to, and for a T5 or T7 partnumber the power and torque too.

When the stock binary is in TxSuite's `Binaries` folder (TxExtras) you can use it:

**Open this file**
: Opens the stock binary.

**Compare to this file**
: Compares the open file to the stock binary, only for a partnumber of the open file's ECU.

**Create new from original**
: Saves a copy of the stock binary under a name you choose and opens it, a clean start for a new tune.

The "..." button shows every known partnumber with a tab per ECU, the open file's ECU selected. The list is grouped by car model and engine type, every group starts folded. The partnumbers that have a stock binary in the `Binaries` folder are coloured. Double-click a row, or select it and click Ok, to look it up. Export saves the list as tab separated text.

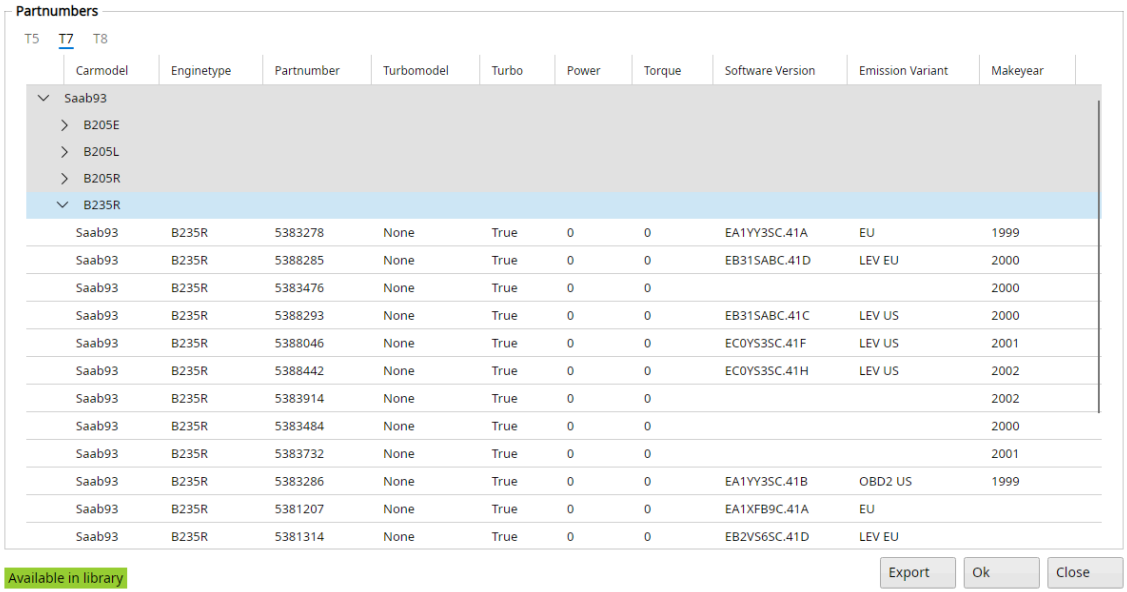



**Image 11: The partnumber list, T7 tab**

## *File → Project*

The Project submenu lets you work on your tune in a project, with roll-back and roll-forward, versions and a logbook. It has a chapter of its own, Projects.

## *File → Exit*

Come on Einstein, what would that do?

Closing TxSuite asks about maps with unsaved changes first, and it won’t close while an ECU is being read or flashed.

# Projects

TxSuite allows you to work on your tune in a project based fashion. Projects have the advantage of having roll-back and roll-forward functions, having versioning and keeping things tidy and together. You are advised to work in a project if you do more than simple easy-tune your binary file.

A project is a folder with a copy of its binary, the transaction log of every change made to it, a logbook and the backups. The projects are in File → Settings → Common → Project folder, the same folder for all three ECU's. Every time a project opens TxSuite makes a backup of its binary first.

All project items are in the File → Project submenu.

## *File → Project → Create a project*

This option lets you create a new project from the binary you have open. The dialog is filled in from the open file, the binary is copied into the new project folder.

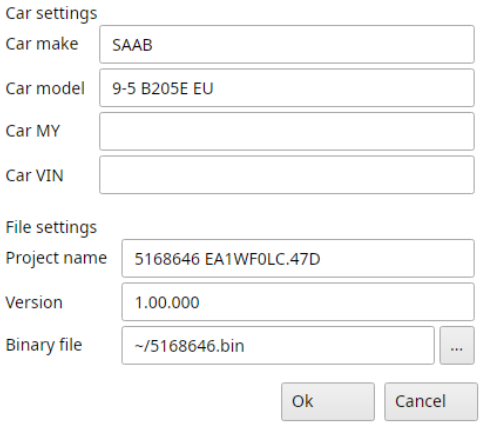



**Image 12: Create a project, filled in from the open file**

## *File → Project → Open a project*

Lets you select and open a previously created project. The list shows each project's number of backups and transactions, when it was last changed and its version. If no projects are available (none were created before) TxSuite will notify you with a message box.

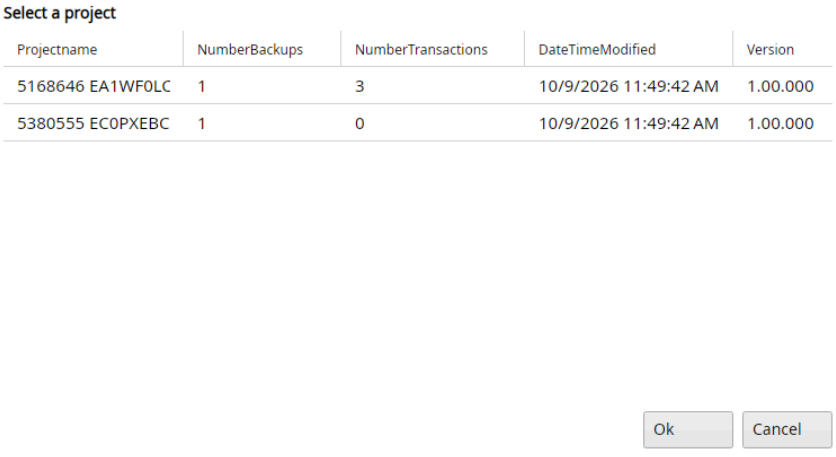



**Image 13: Open a project**

Opening a plain file closes the project. With File → Settings → Common → Auto load last file on startup the last project reopens when TxSuite starts.

## *File → Project → Close project*

Closes the project and allows you to work on single files again.

## *File → Project → Show transaction log*

Shows the transaction log for the current project. Since a project contains only one binary file, you could see this as an undo/redo list for the project binary file. Each row is one write to the binary: when, which symbol, your note, the address and length, and whether it was rolled back. You can roll back or roll forward any entry from here, and type a note for it.

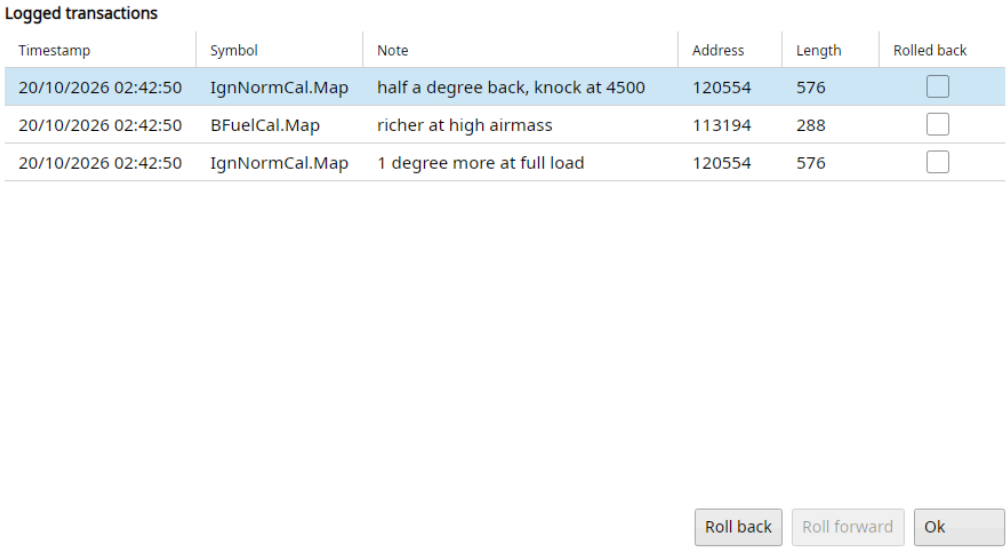



**Image 14: The transaction log after three map saves**

When the log grows past 2000 entries TxSuite offers to purge it.

With File → Settings → Common → Request project notes on, TxSuite asks you for a note every time a change is saved in a project. This way, you can keep track of changes very easily.

## *File → Project → Roll back/undo*

Lets you rollback the last transaction made to the binary file. If no transactions are available to rollback, the item will be disabled.

## *File → Project → Roll forward/redo*

Lets you roll forward (redo) the last transaction that was undone/rolled back. If no transactions are available to roll forward, the item will be disabled.

## *File → Project → Rebuild file*

This enables you to rebuild a project file (binary) up to a certain point in time. TxSuite will ask you for a date and it will restore (if possible) the file that you had at that specific date. Store result as current project file puts it in the project, otherwise TxSuite asks where to save it.

## *File → Project → Edit project*

Lets you edit the properties of the open project. This enables you to manage your version numbering yourself. Version numbers are stored, together with all other project properties, in the xml file in the project folder. Renaming the project renames its folder too.

## *File → Project → Add note to project log*

You can add a note to your project with a timestamp attached to it, so you can keep track of changes you made in your setup. For example, if you start using different injectors or mount a bigger intercooler, you can enter a note of this into the project log for later reference.

## *File → Project → Show project logbook*

Shows you in detail what has happened in your project.

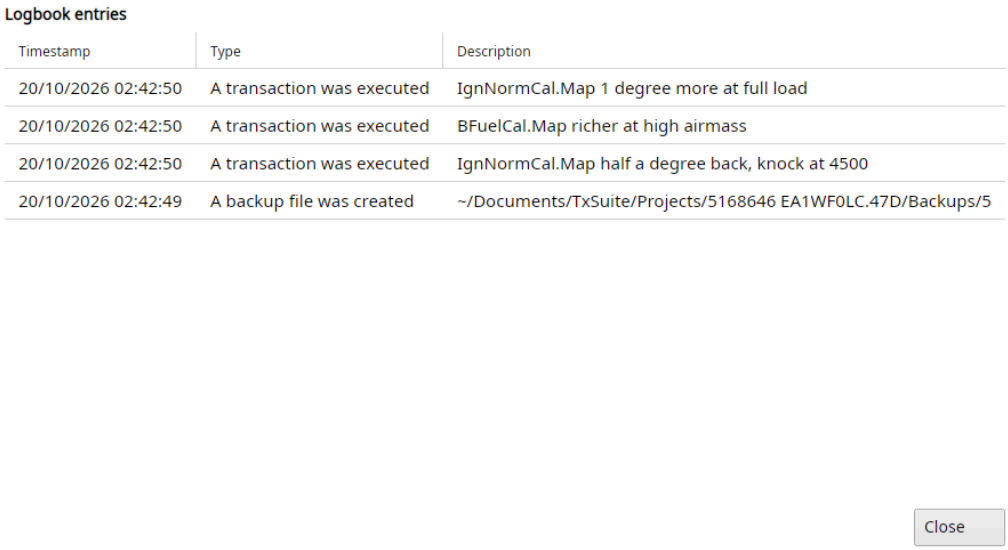



**Image 15: The project logbook**

## *File → Project → Produce latest binary*

Lets you export the project binary in its current state so you can save it in another location easily.

# Settings

File → Settings opens one dialog for every ECU, with the tabs Common, Logging, T5, T7 and T8. Common and Logging hold what every ECU has, each ECU's tab the options only that ECU has. Ok applies every tab at once, also to a file that is open.

Hover over an option for a tip on what it does.

This chapter has the Common and Logging tabs, the T8 tab is in the next chapter. The T5 and T7 tabs are in their manuals.

## *Settings → Common*

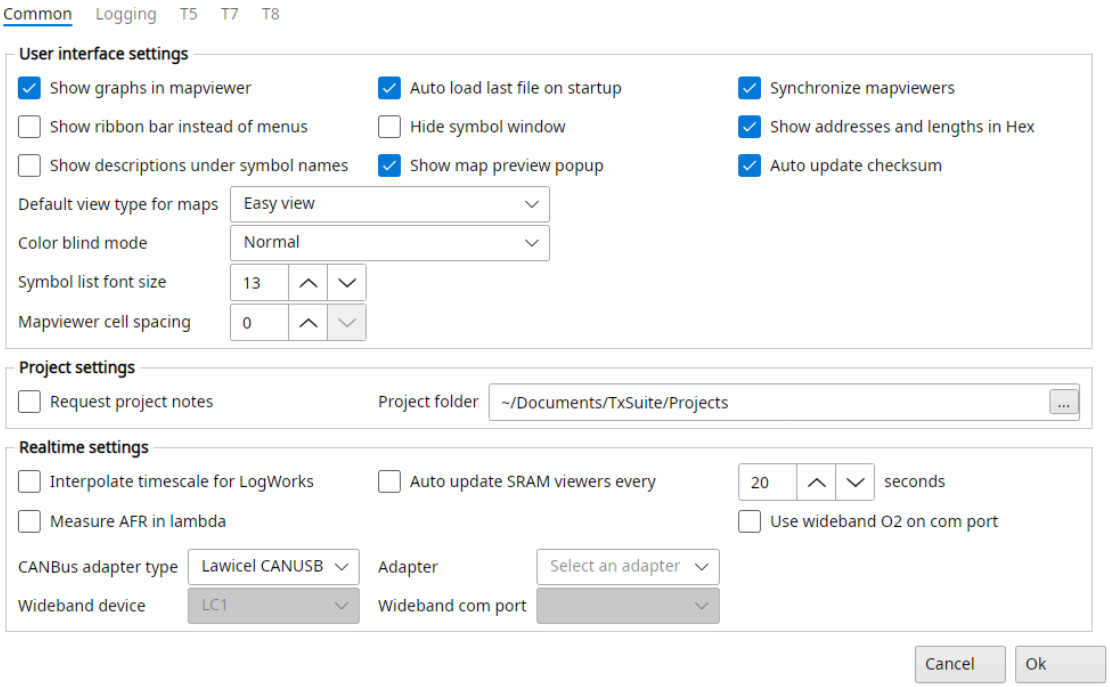



**Image 16: File → Settings, the Common tab**

### *User interface settings*

**Show graphs in mapviewer**
To gain performance in lightweight computers you can switch off the graphical display in the map viewer altogether with this option.

**Show ribbon bar instead of menus**
Shows the menus as a ribbon bar at the top of the window, as in the old suites, see The main window.

**Show descriptions under symbol names**
The symbol list shows each symbol's description on a line of its own under its name, as the old T7Suite and T8Suite did.

**Auto load last file on startup**
If you work on one file for a prolonged period of time you might want TxSuite to reopen the last file automatically when you start it. If the last opened item was a project, TxSuite will reopen that project for you at start-up.

**Hide symbol window**
To gain more space on the main screen you can hide the symbol list to a tab at the side of the window. The tab brings the list out again, and it slides away when you open a map.

**Show map preview popup**
Pointing at a symbol's name in the symbol list shows its map as a read-only table in a popup.

**Synchronize mapviewers**
Map viewers that show the same map follow each other: selecting cells or turning the 3D graph in one does the same in the others.

**Show addresses and lengths in Hex**
TxSuite can display addresses and lengths of symbols in decimal form or in hexadecimal form. This option allows you to switch between the two, in the symbol list, the search results and the compare results.

**Auto update checksum**
Having this checked will rid you of the constant worry whether or not the checksum of your file is valid. Advised is to keep this turned on! With it off TxSuite asks before it corrects a wrong checksum, and before it flashes a file with one. Map saves always update it.

**Default view type for maps**
The view type a map opens in: Hexadecimal view, Decimal view or Easy view. Easy view is best if you don't know what this means ;)

**Color blind mode**
A colour scale for maps and their graphs that is easier to tell apart with colour blindness: Normal, Universal, Protanopia, Tritanopia or Deuteranomaly.

**Symbol list font size**
The size of the text in the symbol list, 8 to 24 pixels.

**Mapviewer cell spacing**
Space between the cells of a map table, 0 to 20 pixels. At 0 they touch as in the old suites.

### *Project settings*

**Request project notes**
If you are working in a project and you save changes to the binary file, TxSuite will prompt you for a note to describe the change made. If disabled, the prompting will be skipped.

**Project folder**
Lets you decide where to store projects (the project root folder), for every ECU. You can use this if you want all the projects to be located on a separate disk partition for example.

### *Realtime settings*

**Interpolate timescale for LogWorks**
LogWorks uses a fixed interval and the frame rate TxSuite reaches differs most of the time. Check this option and TxSuite will interpolate the values to match the interval LogWorks works with, when exporting a logfile to LogWorks.

**Measure AFR in lambda**
If you prefer AFR measurements in lambda values instead of AFR values, enable this.

**Auto update SRAM viewers every … seconds**
If you have map viewers open that show the ECU's RAM, TxSuite can refresh them every 5 to 60 seconds while connected. This enables you to keep an eye on for example the knock map.

**Use wideband O2 on com port**
Reads a wideband controller on a serial port while the realtime panel runs. Its reading is used as the realtime AFR and is shown and logged as Wideband. It overrides the wideband through an ECU symbol on the ECU's tab.

**CANBus adapter type**
Lets you choose which CAN bus interface you have: Lawicel CANUSB, Kvaser HS, CombiAdapter, OBDLink SX, OBDLink EX, J2534, Just4Trionic, SLCAN or ELM327.

**Adapter**
Which adapter of that type to use. For an OBDLink, ELM327, Just4Trionic or SLCAN adapter this is its serial port. The CombiAdapter is found by itself, so the list stays empty for it.

**Serial speed**
The speed of the serial connection to an ELM327 or a Just4Trionic. It shows for SLCAN too, but SLCAN adapters always use 3000000 baud. The OBDLink SX and EX pick their speed themselves.

**Wideband device**
The type of wideband controller on the wideband com port: PLX, LM1, LC1, LM2, ZT2, AEM, STAG or LambdaShield.

**Wideband com port**
The serial port the wideband controller is connected to.

## *Settings → Logging*

TxSuite writes log files of what it does, a file per log and per day. They are what to post when something doesn't work.

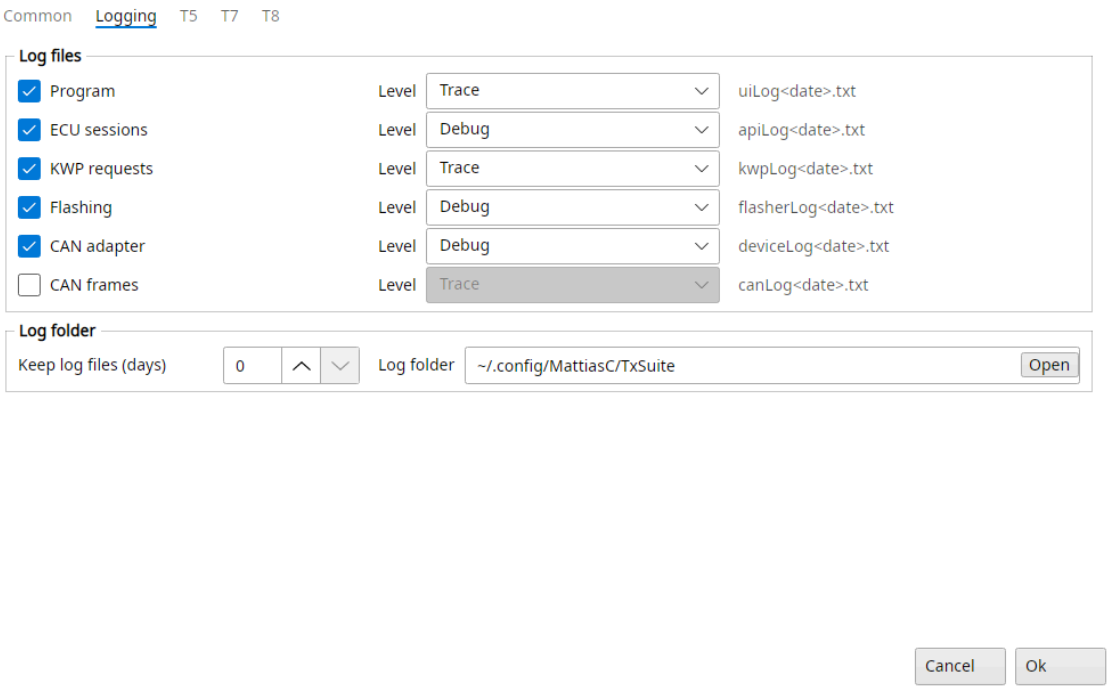



**Image 17: File → Settings, the Logging tab**

Each log can be turned on or off and gets a level of its own. Trace logs everything, Debug a little less, then Info, Warn and Error. A level also logs every one after it, so Error logs the errors alone.

**Program**
What TxSuite and its T5, T7 and T8 parts do: opening and parsing binaries, symbols, checksums, settings, the realtime panel.

**ECU sessions**
The T5, T7 and T8 sessions: connecting, security access, reading and writing memory, fault codes.

**KWP requests**
Every KWP2000 request to a T7 and its answer.

**Flashing**
Reading and flashing an ECU's flash, block by block.

**CAN adapter**
The adapter's driver: opening it, its settings, the frames it couldn't send, its errors.

**CAN frames**
Every CAN frame sent and received. It grows fast and slows the communication down, for finding communication problems only. Off by default, it is the old suites' Enable canbus logging.

**Keep log files (days)**
Log files last written longer ago than this many days are deleted when TxSuite starts and when the settings are saved. 0 keeps them all.

**Log folder**
Where the log files are, its Open button opens the folder.

# Trionic 8 settings

The T8 tab of File → Settings has the options only a T8 has.

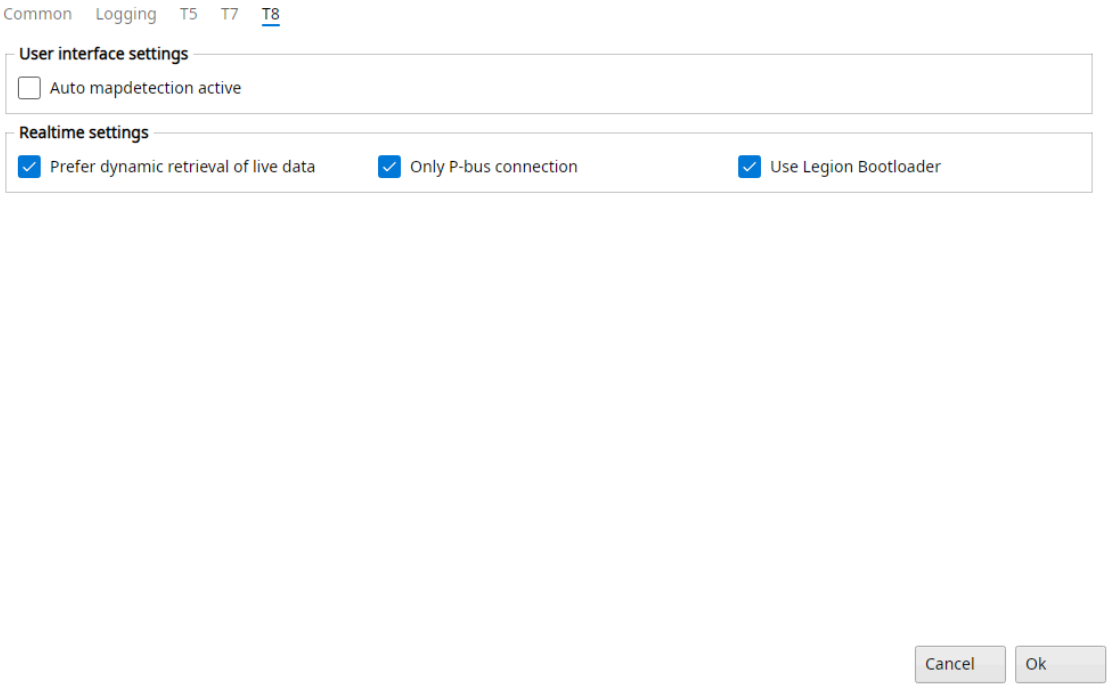



**Image 18: File → Settings, the T8 tab**

## *Settings → T8*

### *User interface settings*

**Auto mapdetection active**
When checked, TxSuite will try to add names to symbols in binaries without symbol tables. This is a guesstimate routine.

### *Realtime settings*

**Prefer dynamic retrieval of live data**
Massive boost to sample rate if the binary is compatible. The realtime panel asks the ECU for all its symbols in one list instead of one by one. If the ECU doesn't take the list, TxSuite reads them one by one for the rest of the session.

**Only P-bus connection**
If checked, TxSuite connects to the ECU on the P-bus (500 kbit/s) only. If not checked, the Lawicel CANUSB, Kvaser, Just4Trionic and SLCAN adapters connect on a slower bus instead, and only some of them fall back to the P-bus when they find no traffic there.

**Use Legion Bootloader**
When checked, TxSuite reads, flashes and recovers the ECU with the Legion bootloader, which only erases and writes the partitions that differ from the file. If not checked, the default bootloader is used.

# Actions menu

The Actions menu has the checksum, the information about the open file, the compare and transfer tools and the SRAM snapshots. Its last section, the tools, is different for every ECU and is described in the chapters after this one.

## *Actions → Verify checksum*

Checks both checksum layers of the open file, the same way as when it opens. The status bar shows the result, “Checksum: OK” or which layer is invalid. With File → Settings → Common → Auto update checksum on a wrong checksum is corrected, otherwise TxSuite asks per layer.

## *Actions → Firmware information*

Lets you see and alter the settings for the current binary file. It has a chapter of its own, Firmware information.

## *Actions → VIN decoder*

Lets you decode SAAB VIN numbers into readable format, from car model and engine type to gearbox, and checks the VIN’s check digit.

It opens with the VIN stored in the open file.

Type a VIN and press Enter or click Decode.

The picture is the stock T7 binary 5168646, it carries a placeholder VIN instead of a car’s. That’s why it decodes as Unknown with a checksum error.

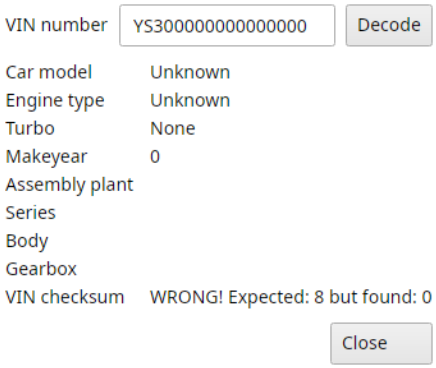



**Image 19: Actions → VIN decoder**

## *Actions → Browse axis information*

Shows you the relations between maps and axis symbols. This is very informative to browse and learn. Double-click a row to open its map, or the axis you clicked on. The symbol menu’s Browse axis information opens it at that symbol.

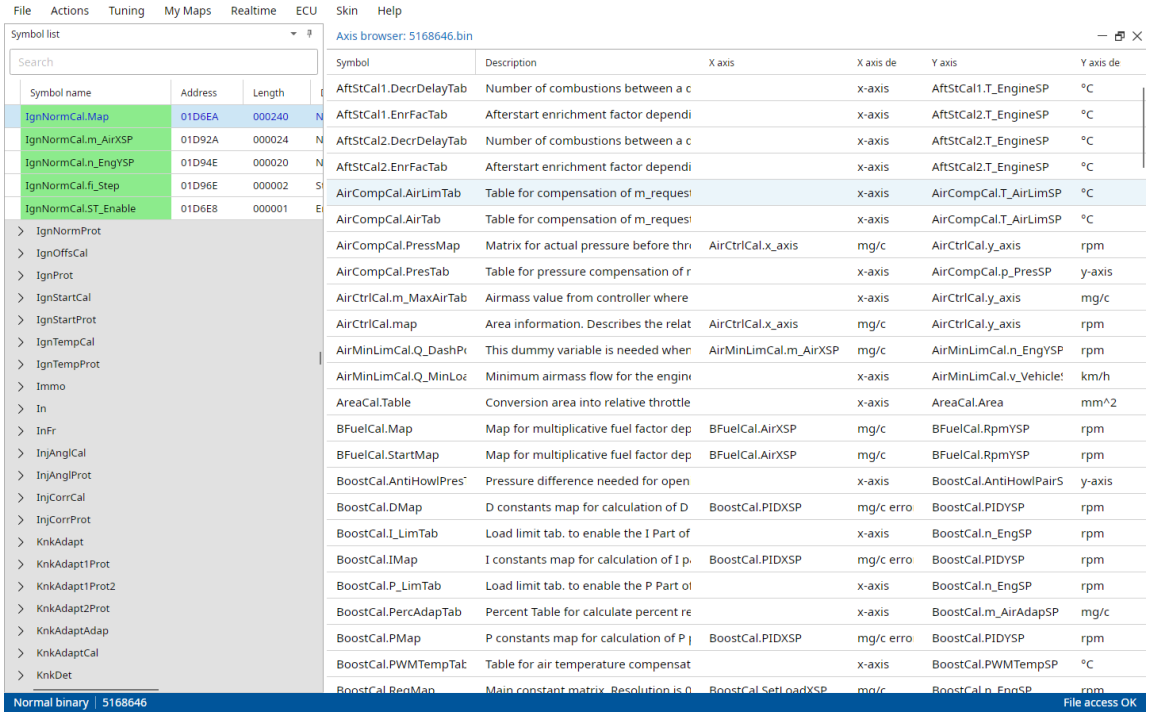



**Image 20: Actions → Browse axis information**

## *Actions → Show interrupt vectors*

This is a little tool for techies.

It shows the start-up vector information for the Motorola MC68377 microprocessor.

## *Actions → Show disassembly*

Lets you disassemble the binary file into human readable text format also known as assembler language. If you're not sure what that means, you don't need this.

The listing is saved next to the bin as `<bin>.asm` and opens in an editor with search and replace. Next to it are the bin's bytes, the cursor in one follows the other. When the `.asm` file already exists TxSuite asks whether to make it again.

## *Actions → Show full disassembly*

Disassembles the whole file from start to end, code or not, into `<bin>_full.asm`.

## *Actions → Compare symbols with other binary*

The most time-consuming thing is comparing maps for different firmware versions by hand. TxSuite gives you the tools to compare maps in different binaries with a click of the mouse. Comparing binary files (actually the maps inside them) is a very educational activity, you will see which maps are different in for example B234L and B234R engine types.

To do this you must first open the primary binary you want to compare. Then, select Compare symbols with other binary and select the secondary binary you wish to compare the first one with. TxSuite will now display a list of symbols that differ in the selected binaries, grouped by category. The list shows how many values differ, the percentage and the average difference. Symbols that only one of the files has are coloured and say in which file they're missing.

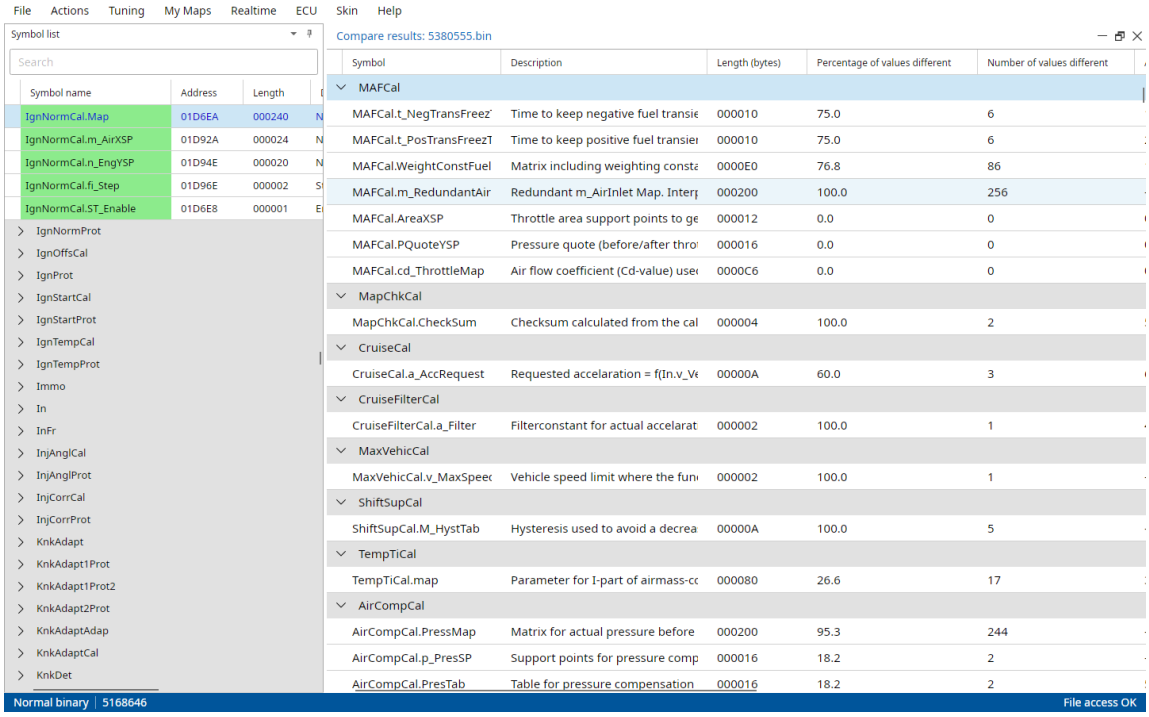



**Image 21: Compare results against another T7 binary**

Double-clicking a symbol in this result list opens its map from both files, in one window with a tab per file. Right-click a symbol for more:

**Open**
The same as a double-click.

**Show differences map**
One map with the difference between the two files.

**Export to CSV**
Saves the result list as a CSV file.

**Export as tuning package**
Saves the selected symbols, from either file, as a tuning package.

## *Actions → Compare binary outside symbolrange*

Compares the bytes of the two files that are not in any symbol, the code and everything else the symbol table doesn't describe.

## *Actions → Compare binary with other binary*

Lets you do a binary (byte-by-byte) compare of two files. This is a good tool to verify whether a programming session was successful or not. Sequences of steps would be:

1. Program ECU
2. Read file from ECU
3. Compare original and downloaded file with Compare binary with other binary.

If it says 0 lines differ the files are identical (successful programming).

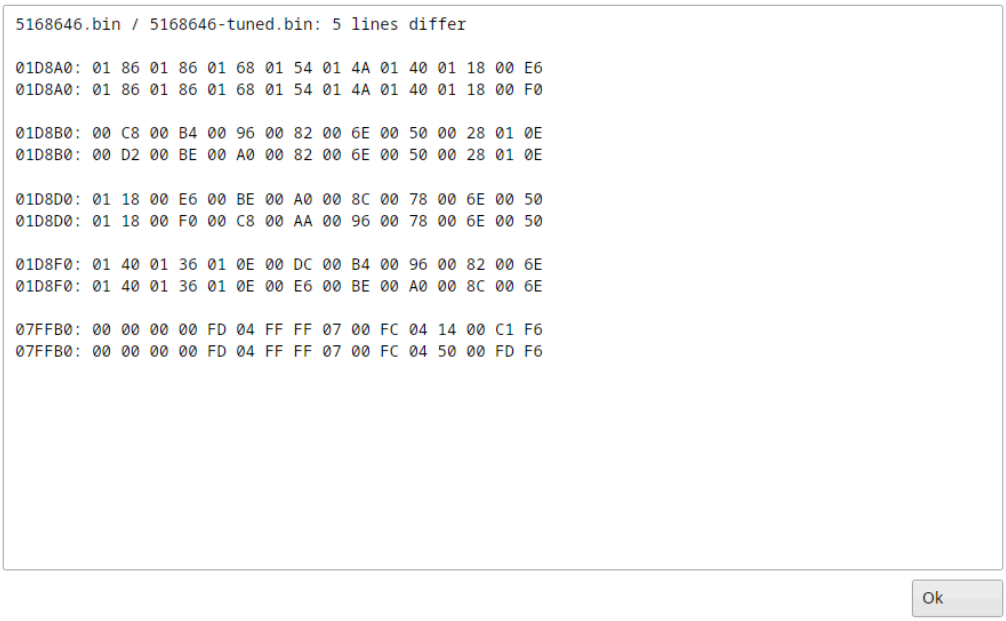



**Image 22: Compare binary with other binary**

## *Actions → Transfer maps to another binary*

If you have a set of tuned maps in one binary file and you want to have them transferred to a different base file, you can use this function. The wizard asks for the target file and shows a list of symbols to choose from, it remembers your last choice. A backup of the target is made before anything is written, and a report shows what was done.

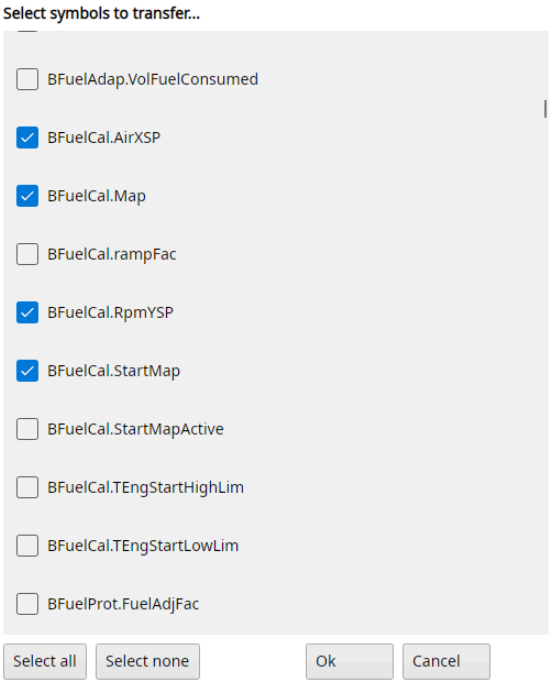



**Image 23: Selecting the symbols to transfer**

## *Actions → Copy address table to another binary*

Some tuning companies choose to damage information inside the binary file to prevent users to have a look at them. Mostly the address table is damaged. With this option you can easily copy the address table from a MATCHING file (original file) to the corrupted file. The target's checksum is updated too. If you don't know what this is about, please don't use it!

## *Actions → Search map content*

Allows you to search maps for a specific value, a number (as shown in Easy view) or a text. It can search the symbol names and descriptions too, and only maps of a given length. The result will be shown in a list, double-click a row to open its map.

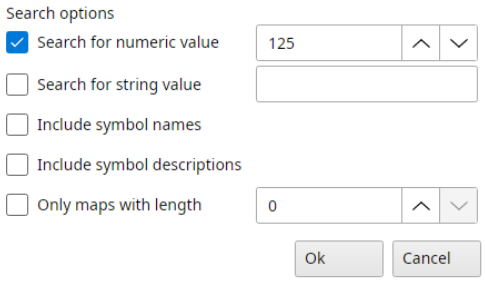



**Image 24: Search map content**

## *Actions → Export map to CSV*

Saves the map selected in the symbol list to a CSV file, which opens in Excel or any other spreadsheet. It replaces the old suites' Excel export. The file is named `<bin>~<symbol>.csv`.

## *Actions → View file in hex*

Lets you explore the binary file in hexadecimal form (advanced users!) The symbol under the cursor is shown. You can change bytes and save them in place, TxSuite makes a backup copy first and asks about unsaved changes when you close it. A loaded SRAM snapshot can be viewed in hex as well.

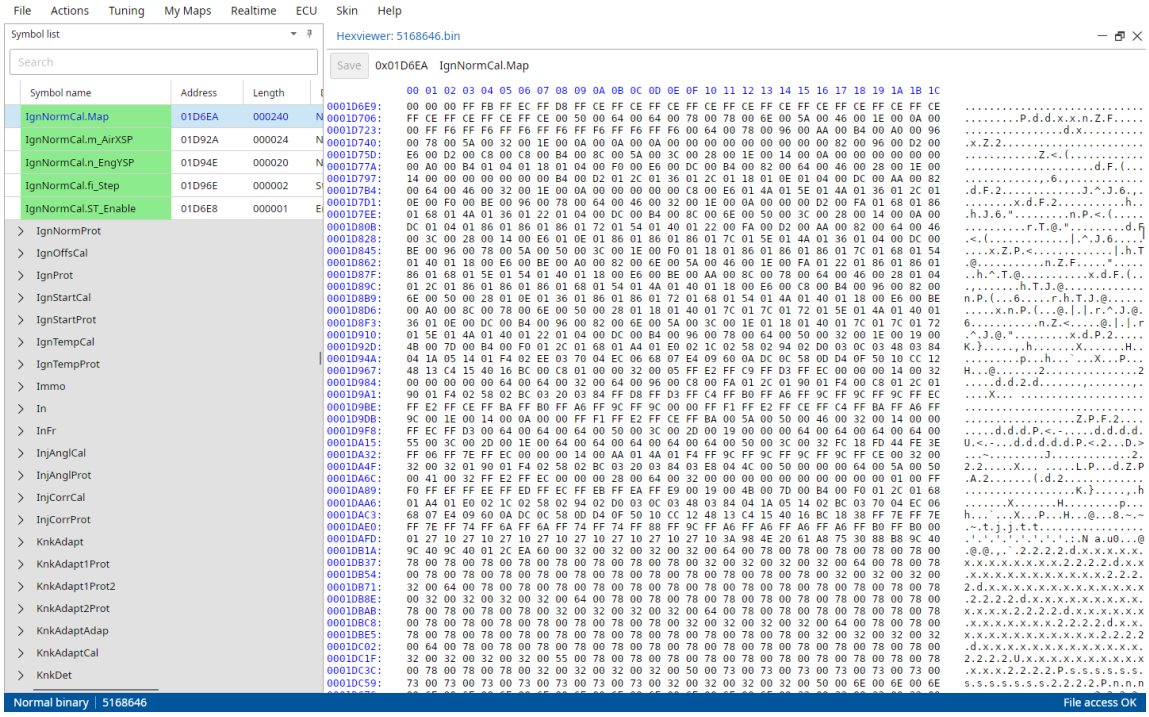



**Image 25: Actions → View file in hex**

## *SRAM snapshots*

An SRAM snapshot is a dump of the ECU's working memory (`.RAM`). It holds the maps that only live in RAM and what the ECU learned (adaption), and while you tune live the changes that aren't in the file yet. ECU → Get SRAM snapshot makes one.

### *Actions → Import SRAM snapshot*

Loads an SRAM snapshot for the open file. Symbols that only live in SRAM then open from it, and the symbol menu's Read from SRAM file shows a map as it was in the snapshot. The status bar shows the loaded snapshot.

# Firmware information

Actions → Firmware information shows the information stored in the current binary file. Ok writes the changes and checks the checksum the way opening a file does, Cancel leaves the file alone. In a project the changes are transactions in the project's log.

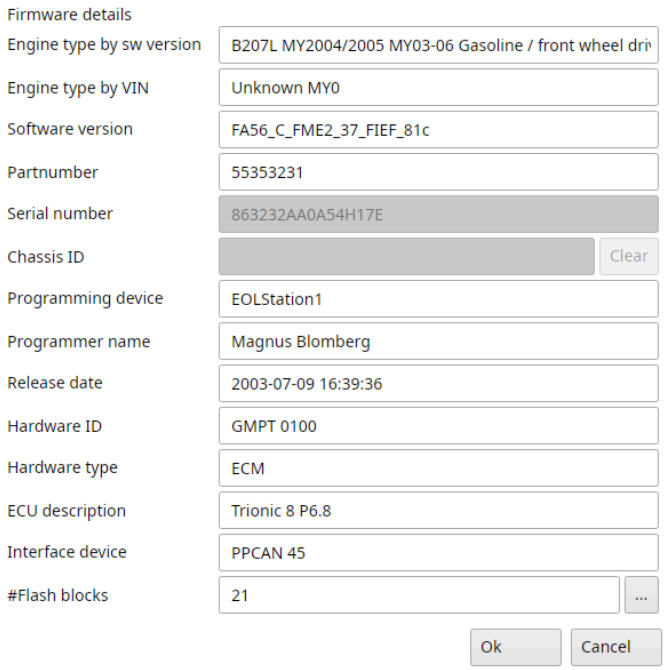



**Image 26: Actions → Firmware information on a T8**

**Engine type by sw version**
The engine, model year and car the software version belongs to.

**Engine type by VIN**
The engine, model year and gearbox decoded from the chassis ID.

**Software version**
The software version of the binary. Double-click its caption to change it.

**Partnumber**
The partnumber of the software.

**Serial number**
The immobilizer code of this binary file. For more information about immo code please refer to the SAAB technical documentation.

**Chassis ID**
Vehicle Identification Number (VIN) for the current binary file. Should match your car's VIN number. Double-click the Chassis ID or Serial number caption to change the VIN and immobilizer code. Clear empties the VIN, Ok writes it. Actions → VIN decoder decodes it.

**Programming device, Programmer name and Release date**
Where, by whom and when the software was made.

**Hardware ID, Hardware type, ECU description and Interface device**
What the software was made for.

**#Flash blocks**
How many valid flash blocks the binary has. The "…" button opens the Flashblocks browser with each block's number, type, address, VIN, ECU type, interface and secret code.

# Trionic 8 tools

The last section of the Actions menu has the tools only a T8 has.

## *Actions → Airmass result viewer*

TxSuite incorporates a function to verify the final (estimated) airmass results based on the most common airmass and torque limiters. The result from these calculations are shown in a table that also shows the limiter that is holding more airmass per combustion back.

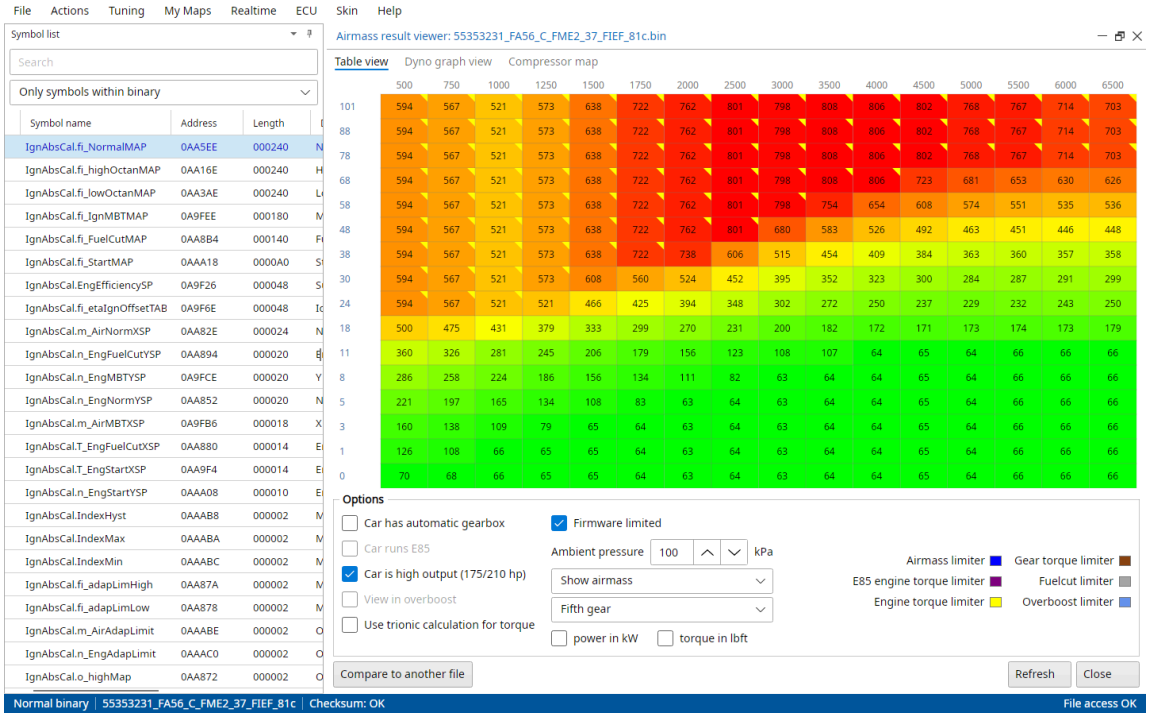



**Image 27: The airmass result viewer's table on a stock T8 binary**

The table has a column per rpm and a row per pedal position, wide open throttle on top. On a T8 the pedal map is a torque request, which is turned into airmass through TrqMastCal.m_AirTorqMap. The coloured triangle in a cell's corner is the limiter that holds it back, the legend says which colour is which. Click a legend entry to open the maps of that limiter. The values can be shown as airmass, torque, horsepower, injector duty cycle, target lambda or AFR, EGT or fuel flow. Power in kW and torque in lbft switch the units of the table and the dyno graph.

The options change the calculation: automatic gearbox, E85, overboost, the gear, the ambient pressure, Trionic's own torque calculation, whether the firmware limits it, and "Car is high output (175/210 hp)" for the high output torque tables. Compare to another file draws a second file over this one, Refresh calculates again after you changed a map. Maps the binary doesn't have don't limit anything.

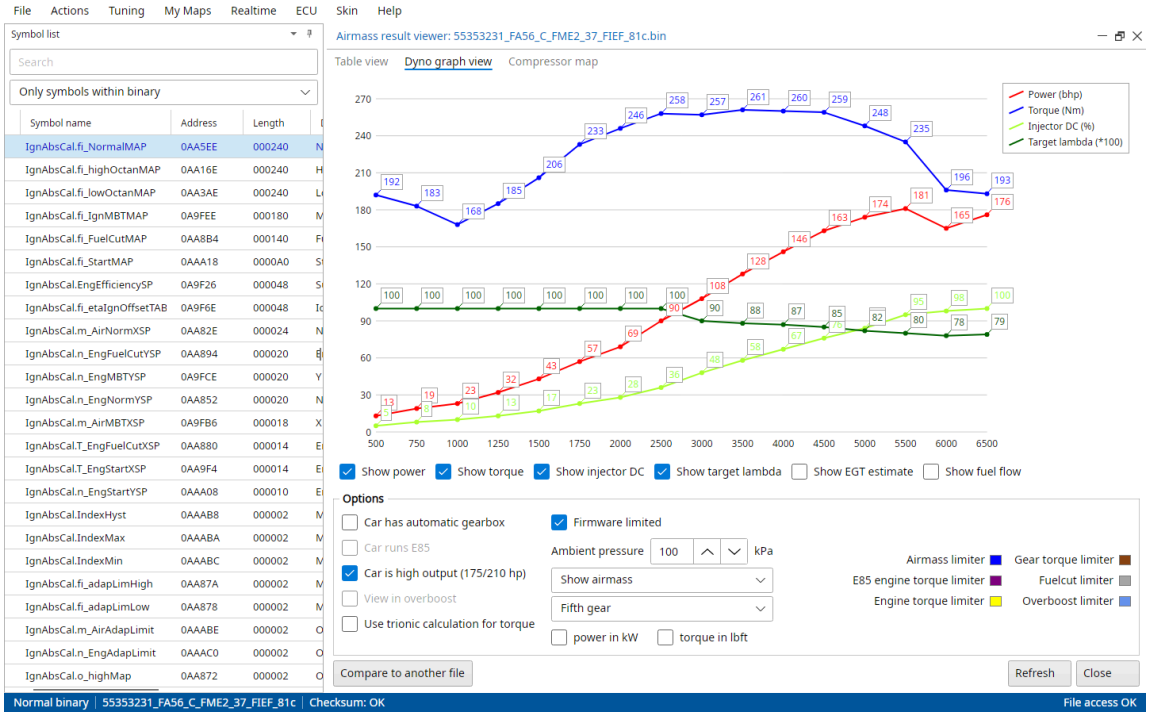



**Image 28: The dyno graph tab**

The Dyno graph view tab draws the result at wide open throttle as power and torque curves, optionally with injector duty cycle, target lambda, the EGT estimate and fuel flow.

The Compressor map tab shows where you will land in the compressor map of your turbo with these airmasses. TxSuite guesses the turbo from the VIN, Select your turbo picks another, and the intake temperature and volumetric efficiency can be set.

## *Actions → TEM editor*

Edits the binary's TEM table, a list of short labels (max 4 characters) each with the symbol it shows. Only enabled when the binary has a TEM table, none of the stock binaries has one. Ok writes every row back into the binary and checks the checksum.

## *Actions → PID editor*

Edits the binary's PID table: which symbol the ECU gives out under which PID (a 16-bit hex number), and whether it can be read and written (Disabled, Enabled, Secured). Rows with something wrong are coloured, a symbol in flash can't be writable for example. The Find box filters the rows. Ok writes every row back into the binary and checks the checksum. If you don't know what this is about, please don't use it!

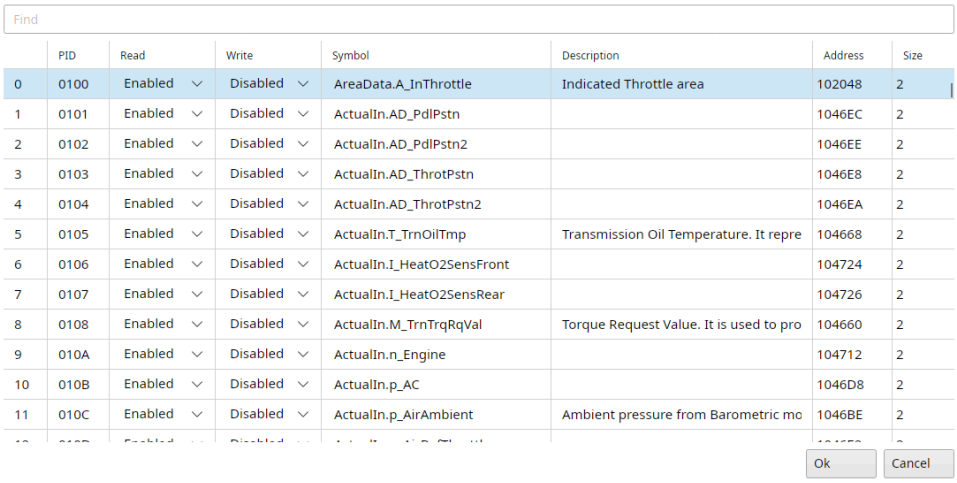



**Image 29: Actions → PID editor on a stock T8 binary**

## *Bit mask symbols*

Symbols that are a set of on/off bits (the Bitmask column in the symbol list) open in “Bit masked view of symbol” instead of a map viewer. It shows the 16 bits, each named by the symbols at the same address, and you tick them on or off. Ok writes the word into the file, or into the ECU for a symbol that lives in SRAM.

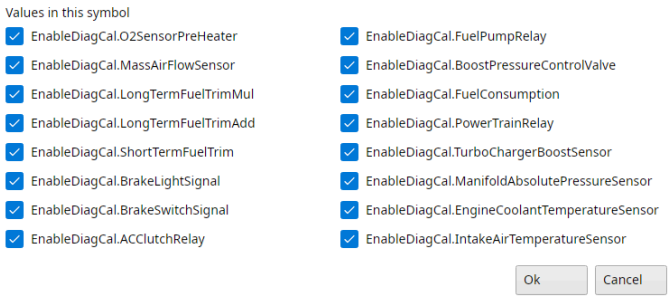



**Image 30: Bit masked view of symbol**

# Tuning menu

The Tuning menu starts with the Tuning Wizard, the rest gives you direct access to the most important maps, grouped in submenus. Each map entry opens that map of the open file in a map viewer. An entry only shows when the binary has the map, so the E85 limiters only show on BioPower software and the jerk maps only where the software has them. Point at an entry to see which symbol it opens.

## *Tuning → Tuning Wizard*

Lets you apply a pre-defined set of Tuning Actions to your binary file. The Tuning Actions are the signed `.t8x` packs that come with TxSuite (its `TuningPacks` folder), the wizard only offers the ones that fit your software.

1. Read the welcome page and click Next.
2. Select the Tuning Action you want from the list, with its software version and author, and click Next.
3. Some packs need a code, visit trionictuning.com and learn how to earn the code.
4. The last page says what will be done. The Tuning Action cannot be reverted, tick “I fully understand the consequenses.” and click Finish.

TxSuite makes a backup of the binary first, `<bin>-<time>-BACKUP-BEFORE-WIZARD-<pack>.bin`. Then it applies the pack as File → Import tuning package does, writes the programmer name and release date into the binary and shows the pack’s message.

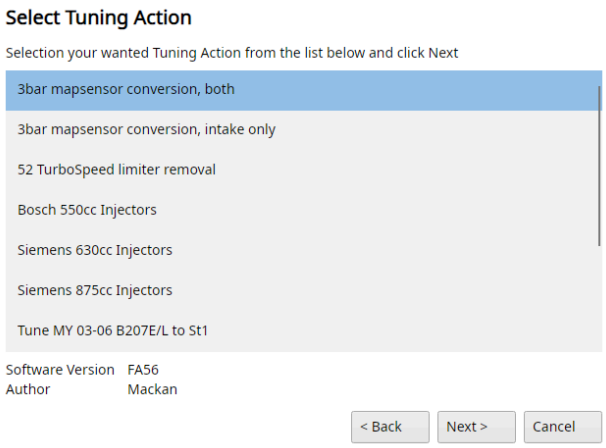



**Image 31: The Tuning Wizard, the Tuning Actions that fit the binary**

## *Tuning → Airmass*

**Max airmass map (manual) and (auto)**
: The maximum airmass limiter (BstKnkCal.MaxAirmass and MaxAirmassAu).

**Airmass Fuelcut**
: The fuel cut limiter, running more airmass per combustion than this value results in a complete shutoff of fuel.

**AirCtrlCal.AirmassLimiter and AirCtrlCal.PRatioMaxTab**
: More airmass limits.

## *Tuning → Torque*

**Nominal torque map**
Converts airmass per combustion to estimated torque (TrqMastCal.Trq_NominalMap).

**Airmass torque map**
Converts torque to airmass per combustion (TrqMastCal.m_AirTorqMap).

**The torque limiters**
Ambient pressure trq limiter, Trq limit in overboost, Trq limit manual 150hp and 175+hp, Trq limit auto 150hp and 175+hp, Manual gear trq limit, and on BioPower software the E85 limits.

**RPM limiter**
The engine speed (RPM) limiter.

## *Tuning → Injectors*

**Inj. Constant**
The injector constant, to compensate for different injectors.

**Inj. dead time and Inj. dead time (Y)**
The battery voltage correction of the injection time and its voltage axis.

## *Tuning → Fuel*

**Fuel correction map, Enrichment Petrol and Enrichment E85, Knock fuel map, Injection end angle map, Jerk enrichment petrol and Fuelmaster**
The fuel maps.

**The ST_Enable switches**
Purge, closed loop (LambdaCal), fuel cut, flex fuel, fuel dynamics and temperature compensation on or off.

## *Tuning → Boost*

**Boost regulation map**
The boost regulation map (AirCtrlCal.RegMap).

**P factor, I factor and D factor**
The P, I and D factors for the PID controller that handles boost.

**The ST_enable switches**
Boost control and the boost, frompressure and area adaptions on or off.

## *Tuning → Ignition*

**Normal ignition map, High octane map, Low octane map**
The ignition maps, the ECU moves between them by octane.

**MBT ignition map, Fuel cut ignition map, Startup map**
The ignition for best torque, during fuel cut and while starting.

**IgnAbsCal.ST_EnableOctanMaps**
Switches the octane maps on or off.

## *Tuning → Pedal*

**Pedal position map and Torque request map**
How the pedal position becomes a torque request.

# My Maps menu

My Maps is your own menu of the maps you use most, so they are one click away. Each ECU has its own, the menu shows the open file's ECU's maps.

## *My Maps → Define myMaps...*

Opens the list of your maps, a row per map with its category, title and symbol. Add and Remove change the list, Ok saves it. The first time the list starts with the old suite's defaults for that ECU.

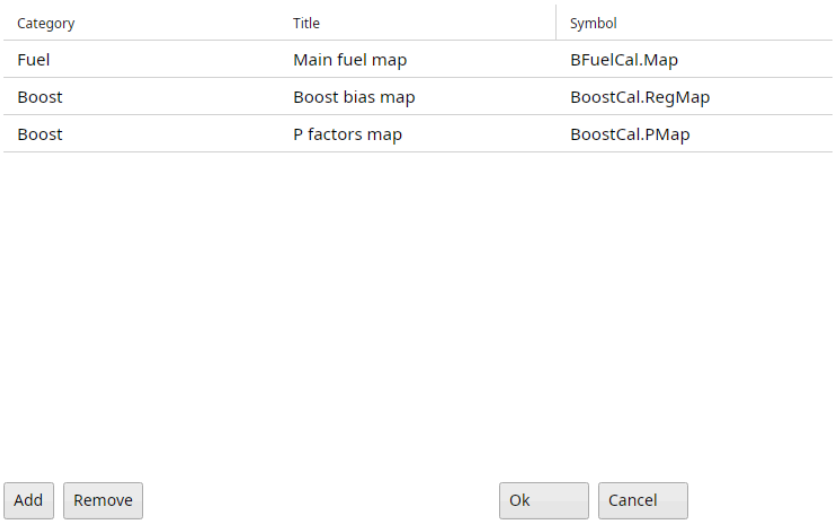



**Image 32: My Maps → Define myMaps..., the T7 defaults**

Below Define myMaps... the menu has a submenu per category, with its maps. A map entry opens that map of the open file.

The symbol menu's Add to MyMaps adds the selected symbol to the list.

The list is kept per ECU in `mymaps.xml` in TxSuite's settings folder, the first start takes over the old suites' lists.

# Realtime menu

The Realtime menu connects to the ECU, runs the realtime panel and works with its logfiles. The ECU must be connected through one of the interfaces in Supported interfaces, set up in File → Settings → Common. A file of the ECU must be open, it tells TxSuite which ECU it's talking to.

The items only your ECU has are described in the next chapter.

## *Realtime → Connect ECU*

Connects to the ECU, the item then reads Disconnect ECU. The status bar shows the connection. Map viewers opened while connected show the ECU's data, see Maps from the ECU in Working with maps.

## *Realtime → Toggle realtime panel*

Allows you to start a realtime session [Shift+F1]. If all connections are done well, TxSuite should be able to connect to the ECU and show you realtime information. Toggling it again stops the panel.

While the panel runs it logs every value it reads into a logfile next to the bin, one per bin and day: `<bin>-yyyyMMdd-CanTraceExt` with the ECU's log extension.

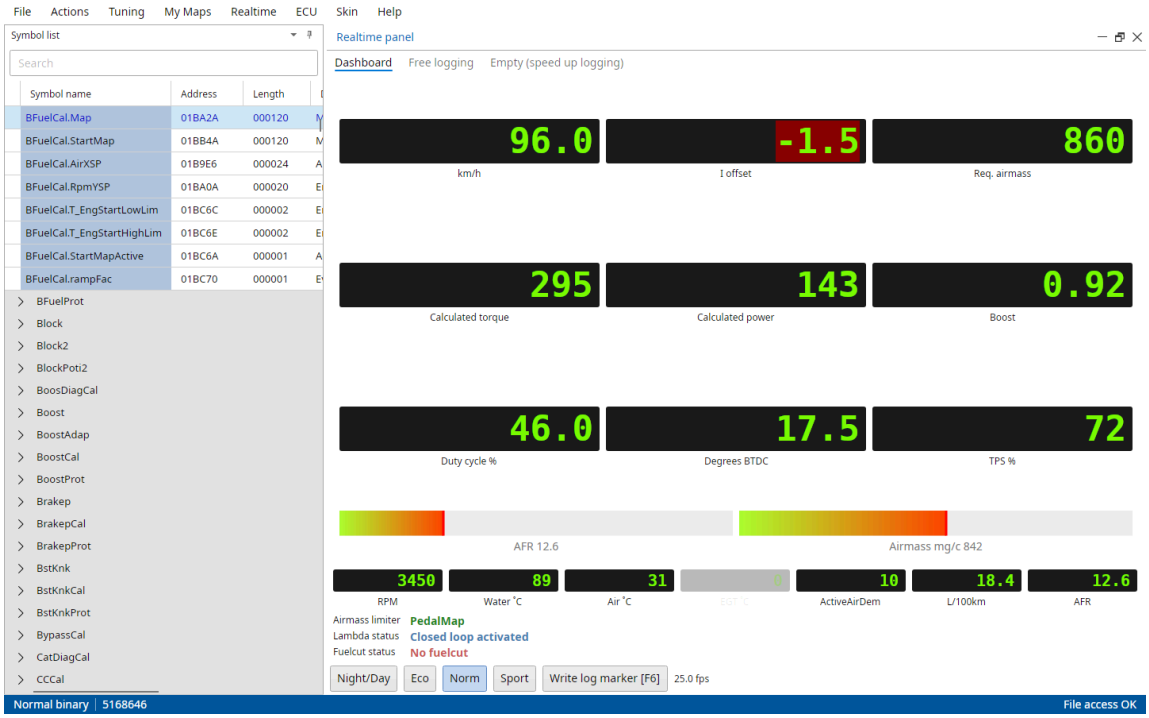



**Image 33: The realtime panel, Dashboard tab**

The panel has three tabs.

**Dashboard**

The most important values as displays: rpm, temperatures, boost, airmass, torque, power, ignition, throttle and more. The AFR display switches between AFR and lambda when you click it. Below are the airmass limiter, lambda status and fuelcut status.

**Free logging**

Your own list of symbols with their value and peak value. Add symbol to list, Edit symbol and Remove symbol (or Del) change it, Ctrl+Up and Ctrl+Down move a symbol up or down. Save layout and Load layout keep a list for later, Reset peak values starts the peaks over. The symbol menu's Add to realtime list adds a symbol from the symbol list.

**Empty (speed up logging)**

Shows nothing, so all the time goes to reading and logging.

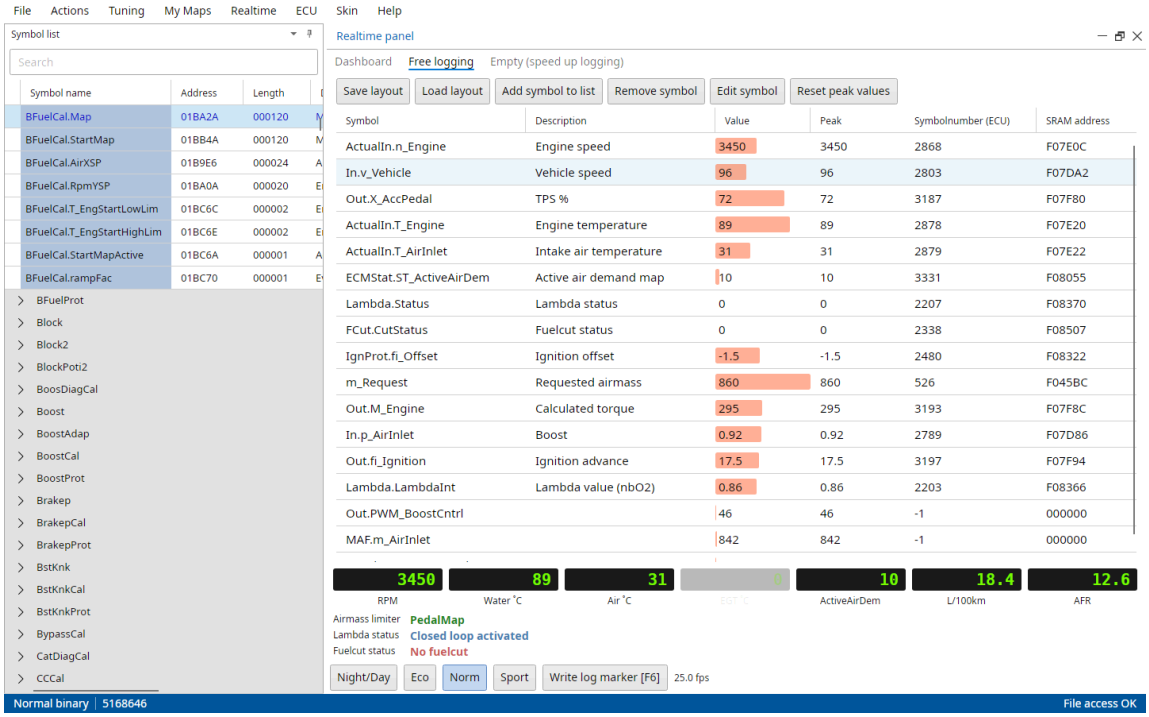



**Image 34: The realtime panel, Free logging tab**

Night/Day switches the panel to dark colours for driving at night. Write log marker [F6] writes a marker into the log.

Open map viewers of the main fuel, ignition and boost maps show the cell the engine is in, in yellow.

## *Realtime → Export logfile to LogWorks*

Will convert and export a logfile to LogWorks format (`.dif`). You choose the symbols and the time range to export in "Logfile data selection". It lists every symbol of the log, all ticked, with Select all and Select none. From time and Up to time start at the beginning and the end of the log. TxSuite doesn't start LogWorks, open the file in it yourself.

## *Realtime → Load logfile*

Loads and shows a logfile in TxSuite's log viewer.

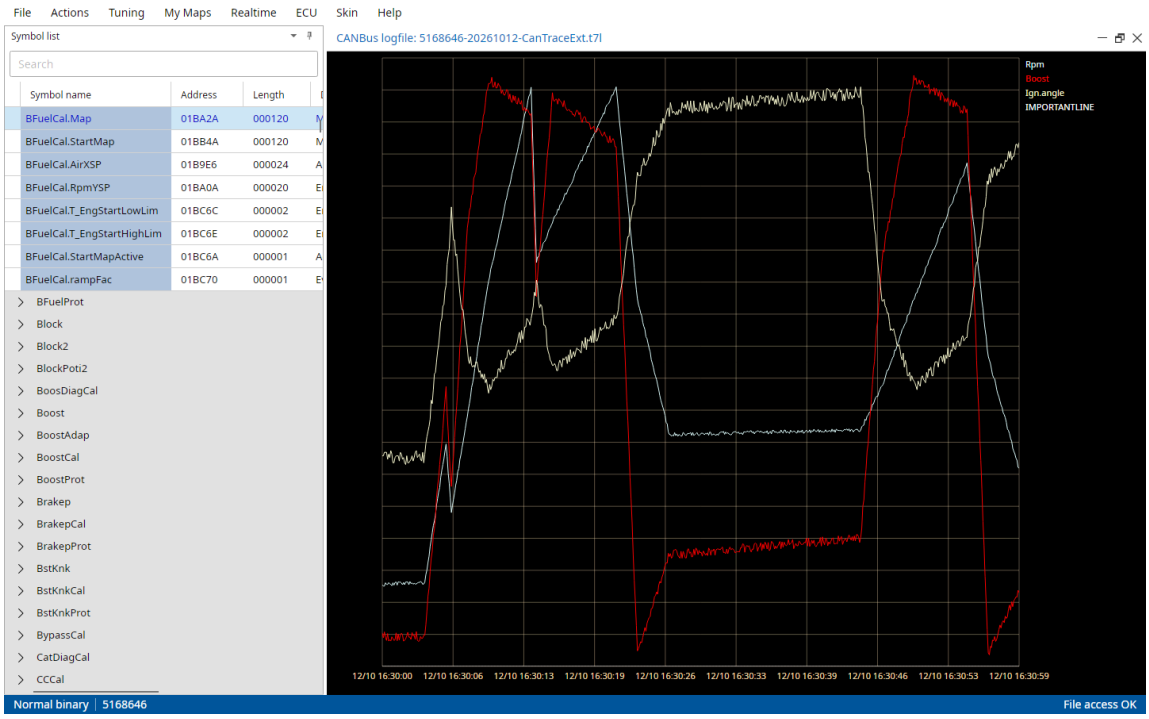



**Image 35: A T7 log in the log viewer**

A log is split into sections where it has a gap of more than 10 seconds, you pick the section first. The channels are drawn over each other, each scaled to its own range, in the colours of Set symbol colors. The legend shows the values under the cursor. The mouse wheel zooms, dragging moves along, and the arrow keys do the same.

## *Realtime → View matrix from logfile*

Lets you view three channels in a logfile displayed as a surface graph (matrix). Choose the symbols for the x axis, the y axis and the value, and whether a cell gets the mean, the minimum or the maximum of its values. The result is a 16 × 16 map between the smallest and largest logged x and y, in a read-only map viewer with its 3D graph.

## *Realtime → Set symbol colors*

Lets you choose the colours that will be used for the symbols in the log viewer. The list has every symbol the ECU has in RAM, with a search box on top.

## *Realtime → Get fault codes*

Allows you to fetch the fault codes (DTC's) from the ECU.

Clear clears the selected fault code, Close closes the window.

On a T8 Clear clears all fault codes in the ECU at once, whichever P code is selected.

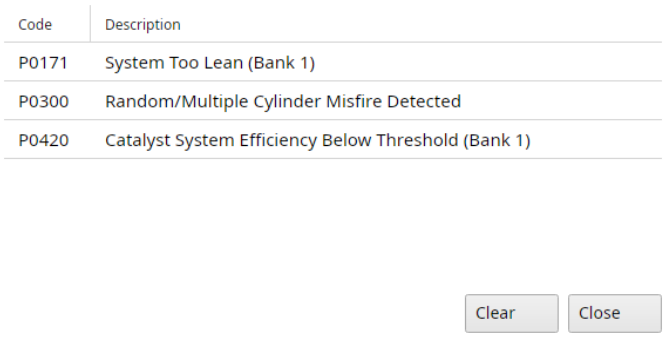



**Image 36: Fault codes**

## *Realtime → Write log marker*

While running a realtime session this enables you to write a marker in the log which you can track back easily later on [F6].

## *Realtime → Export logfile to CSV*

Exports a logfile to CSV, with the symbols and the time range you choose in “Logfile data selection”, as for LogWorks. The first column is the time in seconds since the first line.

# Trionic 8 realtime

The Realtime menu's items only a T8 has.

When connected the status bar shows "Connected:" with the ECU's software version. With File → Settings → T8 → Prefer dynamic retrieval of live data the realtime panel reads all its symbols in one go, which is a lot faster.

## *Realtime → Get ECU information*

Allows you to extract all relevant information from the ECU using the canbus interface: the ECU related data and the calibration data.

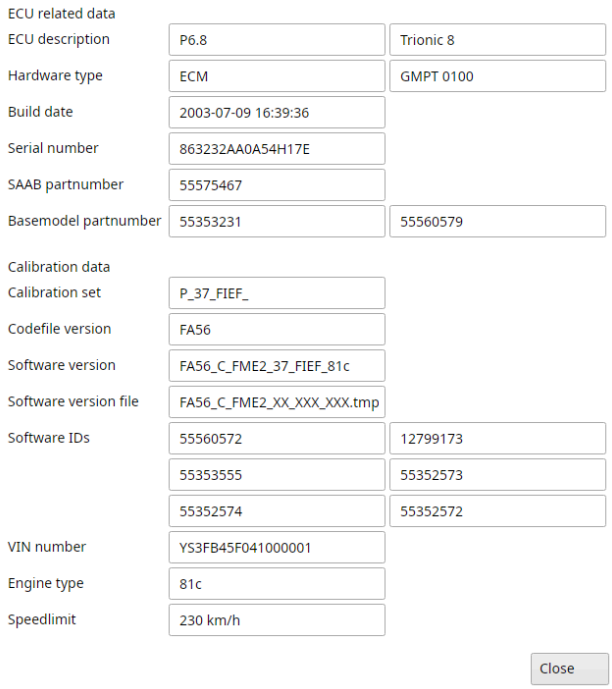



**Image 37: Realtime → Get ECU information**

## *Realtime → Clear DTC and knock counters*

Lets you clear all DTC's and the knock counters in the ECU.

## *Realtime → View knock count map*

If connected to a T8 ECU, this will read the knock counter map from the ECU and display it in a map viewer so you can keep track of where (which load/rpm sites) the system has detected knock.

## *Realtime → View misfire tab*

Reads the misfire counters per cylinder from the ECU and shows them in a map viewer.

# ECU menu

The ECU menu reads and flashes the ECU over CAN, the way the Trionic CAN Flasher does. Like the Realtime menu it needs a file of the ECU open, and an interface set up in File → Settings → Common. The progress shows in the status bar, and TxSuite won't close while it runs.

With File → Settings → T8 → Use Legion Bootloader on (the default) reading, flashing and recovering use the Legion bootloader, which only erases and writes the partitions that differ from the file.

## *ECU → Read ECU*

Downloads the flash content from the connected ECU. It will ask you where to store the binary file.

## *ECU → Flash current file to ECU*

Flashes the open file to the connected ECU. Map viewers with unsaved changes are pointed out first, they are not in the file yet. A wrong checksum is corrected first, or TxSuite offers to correct it when File → Settings → Common → Auto update checksum is off. When it's done TxSuite says whether it worked.

A good check after flashing is to read the ECU back and compare the two files with Actions → Compare binary with other binary.

When the flash was erased but programming failed, TxSuite asks whether to attempt to recover the ECU with the same file.

## *ECU → Recover ECU*

Brings back an ECU whose flashing failed halfway, by flashing a file through the bootloader. It only takes a T8 binary whose checksum verifies.

# Skin menu

TxSuite has a light and a dark skin.

## *Skin → Light, Dark and Follow the system*

Light and Dark pick the skin, Follow the system takes the one your system uses. The choice is the same for every ECU and is remembered.

## *Skin → Cascade windows and Tile windows*

Cascade windows puts the windows in the workspace on top of each other, slightly shifted, so every title bar shows. Tile windows horizontally and Tile windows vertically put them next to each other so they all fit.

## *Skin → Dock all floating windows*

Brings every window that floats outside the main window back into the workspace.

# Help menu

The Help menu has the manuals and documentation of all three ECU's, whatever file is open.

## *Help → T5, T7 and T8*

Each has a submenu with the User manual, this book for that ECU, and the Trionic N documentation, Dilemma's book about that ECU.

## *Help → Release notes*

Opens the releases page with what changed in each version.

## *Help → Check for updates*

Looks for a newer TxSuite and shows the result in the status bar. When there is one it asks, OK downloads the setup on Windows and opens the release page elsewhere. TxSuite also checks when it starts.

## *Help → About TxSuite*

The version, and the people who made the suites possible. TxSuite was created with the help of lots of people on ecuproject.com and trionictuning.com.